

# Eine Einführung in Rust | SA

## Vorbereitung auf die Studienarbeit «GitLab Time-Report»

---

### INHALTSVERZEICHNIS

|                                                       |           |                                                             |           |
|-------------------------------------------------------|-----------|-------------------------------------------------------------|-----------|
| <b>1. Rust Basics</b> .....                           | <b>2</b>  | 9.5. Paths .....                                            | 17        |
| 1.1. Rust Projekt einrichten .....                    | 2         | <b>10. Closures</b> .....                                   | <b>17</b> |
| 1.2. Variablen .....                                  | 2         | 10.1. Funktionstypen .....                                  | 18        |
| 1.3. Numerische Datentypen .....                      | 2         | <b>11. Iteratoren</b> .....                                 | <b>19</b> |
| 1.4. Rückgabewerte .....                              | 2         | 11.1. Häufig verwendete Adapter .....                       | 19        |
| <b>2. Ownership &amp; Borrowing</b> .....             | <b>3</b>  | <b>12. Generics</b> .....                                   | <b>20</b> |
| 2.1. Ownership in Funktionen .....                    | 4         | 12.1. In Structs, Enums und Methoden .....                  | 20        |
| 2.2. Borrowing .....                                  | 4         | 12.2. In Funktionen .....                                   | 21        |
| 2.3. Anfängertipps zum Umgang mit Referenzen ...      | 5         | <b>13. Traits</b> .....                                     | <b>21</b> |
| <b>3. Slices</b> .....                                | <b>5</b>  | 13.1. Traits aus der Standard Library .....                 | 21        |
| <b>4. Strings</b> .....                               | <b>6</b>  | 13.1.1. Beispiel: Display Trait implementieren ...          | 22        |
| 4.1. String vs. &str .....                            | 6         | 13.2. Eigene Traits .....                                   | 23        |
| 4.2. String-Formatierung .....                        | 6         | 13.3. Default Implementation von Traits .....               | 23        |
| 4.3. Auf einzelne Zeichen zugreifen .....             | 7         | 13.4. Traits als Parameter .....                            | 24        |
| <b>5. Compound Types &amp; Collections</b> .....      | <b>7</b>  | 13.4.1. Trait Bound Syntax .....                            | 24        |
| 5.1. Arrays .....                                     | 7         | 13.4.2. Mehrere Trait Bounds .....                          | 24        |
| 5.2. Vector .....                                     | 7         | 13.5. Traits als Rückgabetypen .....                        | 24        |
| 5.3. Tuples .....                                     | 8         | 13.6. Methoden nach implementieren Traits<br>kreieren ..... | 25        |
| 5.4. Hashmap .....                                    | 8         | 13.7. Trait Objekte .....                                   | 25        |
| <b>6. Algebraische Typen</b> .....                    | <b>8</b>  | <b>14. Lifetimes</b> .....                                  | <b>25</b> |
| 6.1. Structs .....                                    | 8         | 14.1. Lifetime Annotations in Funktionen .....              | 26        |
| 6.1.1. Structs ausgeben: Die Display & Debug Traits . | 9         | 14.2. Now you're thinking with lifetimes! .....             | 27        |
| 6.1.2. Methoden in Structs: impl-Blocks .....         | 9         | 14.3. Die static Lifetime .....                             | 28        |
| 6.2. Enums .....                                      | 10        | <b>15. Testing</b> .....                                    | <b>28</b> |
| 6.2.1. Das match Statement .....                      | 10        | 15.1. Tests auf panic!() überprüfen .....                   | 29        |
| 6.3. Beispiel: Super Mario Power-Ups .....            | 11        | 15.2. Tests mit Result<T, E> .....                          | 30        |
| <b>7. Optionale Werte</b> .....                       | <b>12</b> | 15.3. Integration Tests .....                               | 30        |
| 7.1. Simplere Checks mit if-let und let-else ....     | 12        | <b>16. Code-Dokumentation</b> .....                         | <b>31</b> |
| <b>8. Error Handling</b> .....                        | <b>13</b> | <b>17. Smart Pointers</b> .....                             | <b>31</b> |
| 8.1. panic! .....                                     | 13        | 17.1. Box<T> .....                                          | 32        |
| 8.2. Result<T, E> .....                               | 13        | 17.2. Rc<T> .....                                           | 32        |
| 8.2.1. Eigene Error-Typen .....                       | 14        | 17.3. Mutex<T> .....                                        | 33        |
| 8.2.2. Result vs. Option .....                        | 14        | 17.4. Arc<T> .....                                          | 33        |
| 8.2.3. Konvertierung Result ⇔ Option .....            | 14        | 17.5. RefCell<T> und Interior Mutability .....              | 33        |
| 8.3. Kürzeres Error Handling mit unwrap/expect ...    | 14        | 17.5.1. Mehrere Owner von Mutable Daten .....               | 34        |
| 8.4. Error Propagation: Der ? (try) Operator .....    | 15        | 17.6. Referenzzyklen mit Weak<T> vermeiden .....            | 34        |
| 8.5. Beispiel: Grandparent in einem Tree .....        | 16        | <b>18. Macros</b> .....                                     | <b>35</b> |
| <b>9. Projektstruktur</b> .....                       | <b>16</b> | 18.1. Deklarative Makros .....                              | 35        |
| 9.1. Crates .....                                     | 16        | 18.2. Nützliche Makros der Standard Library .....           | 36        |
| 9.2. Package .....                                    | 16        |                                                             |           |
| 9.3. Modules .....                                    | 17        |                                                             |           |
| 9.4. Visibility .....                                 | 17        |                                                             |           |

## 1. RUST BASICS

In diesem Kapitel werden die Basics beschrieben, welche kleinere Unterschiede zu anderen Sprachen aufweisen und deswegen erwähnenswert sind.

### 1.1. RUST PROJEKT EINRICHTEN

Die meisten Funktionen werden direkt über Cargo, den integrierten Package Manager von Rust, ausgelöst.

| Befehl                              | Beschreibung                                                                                                                                                           |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cargo new &lt;name&gt;</code> | Erstellt ein neues Projekt mit den benötigten Dateien und einem Git Repo                                                                                               |
| <code>cargo build</code>            | Startet einen neuen Build. <code>--release</code> entfernt Debuginfos und aktiviert Optimierungen                                                                      |
| <code>cargo run</code>              | Startet einen neuen Build und führt das Projekt danach aus                                                                                                             |
| <code>cargo check</code>            | Überprüft, ob der Code kompiliert, ohne einen Build zu starten                                                                                                         |
| <code>cargo fmt</code>              | Formatiert den Code nach dem Rust Style Guide                                                                                                                          |
| <code>cargo fix</code>              | Wendet alle Verbesserungsvorschläge des Compilers bei Warnungen und Fehlern an                                                                                         |
| <code>cargo clippy</code>           | Lässt den clippy Linter laufen, welcher auf falschen oder unnötigen Code überprüft  |

### 1.2. VARIABLEN

Variablen werden mit dem `let` **Keyword** definiert und müssen immer vor Verwendung initialisiert werden. Sie sind standardmässig immutable (read-only). Um sie mutable zu machen, muss das `mut` **Keyword** verwendet werden.

```
let read_only = 5;
let mut writeable = 10;
writeable = read_only + writeable;
println!("{read_only}, {writeable}");
// Output: 5, 15

let not_initialized;
println!("{not_initialized}"); // Compiler Error
not_initialized = 1;
println!("{not_initialized}"); // Compiler happy
// Output: 1
```

Rust ist statically typed, das heisst jede Variabel muss zur Compilezeit einen Typen haben. Im obigen Beispiel wurde der Typ vom Compiler selbst erkannt («type inference»). Er kann aber auch explizit mit einer **Type Annotation** angegeben werden.

```
let eight_byte_unsigned_int: u8 = 1;
let thirty_two_byte_signed_int: i32 = 1337;
```

Ein weiterer Unterschied zu anderen Sprachen ist, dass durch **Shadowing** eine Variabel innerhalb desselben Scopes neu definiert werden kann, auch mit einem anderen Datentyp.

```
let spaces = "  "; // spaces is a string slice
let spaces = spaces.len(); // spaces has been shadowed, the new spaces is a number
```

### 1.3. NUMERISCHE DATENTYPEN

Bei Numerischen Datentypen wie Integers und Floats muss die Bytegrösse spezifiziert werden. Signed und Unsigned Integers können 8, 16, 32, 64 oder 128 byte gross sein, Floats 32 oder 64 byte. Die Standarttypen sind `i32`, `u32`, `f64` – diese sind für die meisten Dinge in Rust genügend. Floats haben keine unsigned Variante. Zahlen können «\_» als visuellen Seperator und eine Typenangabe haben, wenn gewünscht: `100_000u16`

### 1.4. RÜCKGABEWERTE

Wie in anderen Programmiersprachen kann eine Funktion einen Rückgabetyt haben oder nicht. Hat sie keinen, wird sie in der Signatur weggelassen. In anderen Sprachen wird der Rückgabetyt mit `void` bezeichnet. Rust nennt ihn in solchen Fällen «()» – den **Unit Type**.

```
fn hello_world() {
    println!("Hello World!");
}
```

Ebenfalls bemerkenswert sind die zwei unterschiedlichen Möglichkeiten, einen Wert zurückzugeben. Einerseits wie gewohnt mit dem `return` Keyword oder indem bei der letzten Zeile in einer Funktion das Semikolon weggelassen wird. Dann ist die Zeile nämlich kein **Statement** mehr, sondern eine **Expression**. Statements führen eine Anwendung aus und geben keinen Wert zurück, Expressions werden zu einem Wert evaluiert. Das bedeutet, dass beide untenstehenden Funktionen kompilieren:

```
fn plus_one(number: i32) → i32 {  
    return number + 1;  
}
```

```
fn plus_one(number: i32) → i32 {  
    number + 1  
}
```

Dasselbe gilt innerhalb von Control Flow Blocks, wie ein `if-else`, welche am Ende einer Funktion stehen.

```
fn bigger(x: i32, y: i32) → i32 {  
    if x > y {  
        return x;  
    } else {  
        return y;  
    }  
}
```

```
fn bigger(x: i32, y: i32) → i32 {  
    if x > y {  
        x  
    } else {  
        y  
    }  
}
```

---

## 2. OWNERSHIP & BORROWING

Ownership ist eines der Hauptkonzepte in Rust, welches es von anderen Sprachen unterscheidet. Das Ownership-Modell soll **Sicherheit** garantieren – das **Fehlen von Undefined Behavior**. Der Rust-Compiler garantiert, dass diese drei Regeln immer eingehalten werden.

1. Jeder Wert in Rust hat einen Owner.
2. Es kann immer nur einen Owner zur selben Zeit geben.
3. Wenn ein Owner den Scope verlässt, wird der Wert gelöscht («dropped»).

Diese drei Regeln sorgen dafür, dass Memory Safety auch ohne performance-fressenden Garbage Collector (z.B. in Java, .NET, Python, Go, Haskell...) oder heikles manuelles Allokieren und Freigeben von Speicher (in C/C++) garantiert wird.

Die Regeln kommen hauptsächlich dann zum Einsatz, wenn Werte auf dem Heap alloziert werden<sup>1</sup>, wie beispielsweise String. Ein String speichert den Inhalt des Strings auf dem Heap und legt auf dem Stack einen Pointer auf den ersten Charakter des Strings auf dem Heap, die Länge sowie die Kapazität des Strings ab.

Wird dieser String einer anderen Variabel zugewiesen, zeigen die Variablen nicht wie in anderen Sprachen auf denselben Speicherbereich auf dem Heap. In Rust wird ein **Move** durchgeführt, also der Owner gewechselt. Die alte Variabel hat nun keinen Zugriff mehr auf diese Daten und der Compiler generiert eine Fehlermeldung.

```
let s1 = String::from("Hello!");  
let s2 = s1;  
println!("{s1}");  
// Compiler error: borrow of moved value: `s1`
```

Falls eine Kopie beabsichtigt war, muss der **Wert geklont** werden. Dies entspricht einer Deep Copy in anderen Programmiersprachen. Je nach Grösse des Wertes auf dem Heap kann dies aber performanceintensiv sein.

```
let s1 = String::from("Hello!");  
let s2 = s1.clone();  
println!("{s1}");
```

Dies gilt aber nicht für alle Datentypen: Gewisse Typen implementieren den **Copy Trait**, welcher es erlaubt, Werte zu kopieren anstatt sie zu verschieben. `clone()` hat hier also dieselbe Funktionalität wie eine reguläre Zuweisung.

Dies sind vor allem Typen, welche ausschliesslich auf dem Stack gespeichert werden: Alle Integer- und Floating Point-Typen, `bool`, `char` und Tupel, wenn alle Werte darin ebenfalls den Copy Trait implementieren

---

<sup>1</sup>Korrekt gesagt: Auf alle Datentypen, welche nicht den Copy Trait implementieren

Der Copy Trait erlaubt also folgenden Code ohne Compilerfehler

```
let x = 3;
let y = x;
println!("x = {x}, y = {y}");
```

## 2.1. OWNERSHIP IN FUNKTIONEN

Wird eine Variable einer Funktion übergeben, wird sie ebenfalls kopiert oder verschoben. Dasselbe gilt für den Rückgabetypen.

```
fn main() {
    let s1 = String::from("Hello");
    takes_ownership(s1); // s1 has been moved and is no longer valid after this call
    let s2 = gives_ownership();
    let s3 = takes_and_gives_back_ownership(s2); // s2 has been moved and is no longer valid
    println!("Only s3 is valid. s3 = {s3}");
}

fn takes_ownership(owned_string: String) {
    println!("{owned_string}");
}

fn gives_ownership() → String {
    let new_string = String::from("your string");
    new_string
}

fn takes_and_gives_back_ownership(owned_string: String) → String {
    owned_string
}
```

## 2.2. BORROWING

Weil das Übergeben von Ownership nicht immer nötig ist, gibt es in Rust **Borrowing**. Dabei wird nicht das Objekt selbst, sondern nur eine Referenz davon an eine Funktion übergeben. Referenzen werden mit einem & gekennzeichnet, sowohl in der Methodensignatur, als auch beim Aufruf. Referenzen zeigen immer auf ein valides Objekt.

```
let s1 = String::from("hello");
let length = calculate_length(&s1);
println!("The length of '{s1}' is {length}");
// Output: The length of 'hello' is 5

fn calculate_length(s: &String) → usize {
    s.len()
}
```

Die Referenzen sind standardmässig immutable (read-only), können aber auch mutable gemacht werden, um Parameter modifizieren zu können.

```
let mut s = String::from("Hello");
append(&mut s);
println!("{s}"); // Output: Hello there!

fn append(str: &mut String) {
    str.push_str(" there!");
}
```

Mutable References haben aber eine entscheidende Limitierung: Es können entweder unendlich viele immutable oder eine einzige mutable Reference gleichzeitig existieren. Der folgende Code kompiliert also nicht.

```
let mut s = String::from("Hello");
let r1 = &s; // OK, immutable borrow
let r2 = &s; // OK, multiple immutable borrows are allowed
let r3 = &mut s; // Error: cannot borrow `s` as mutable because it is also borrowed as immutable
println!("{r1}, {r2}, and {r3}");
```

Es gibt also drei Arten, wie ein Wert übergeben werden kann:

- **Ownership übergeben:** Der Callee übernimmt die Ownership vom Caller (value: `i32`)
- **Immutable Borrowing:** Der Callee erhält eine read-only Referenz, der Caller behält die Ownership (value: `&i32`)
- **Mutable Borrowing:** Der Callee erhält eine schreibbare Referenz, der Caller behält die Ownership (value: `&mut i32`)

### 2.3. ANFÄNGERTIPPS ZUM UMGANG MIT REFERENZEN

In Rust ist der Compiler dein bester Freund. Die Fehlermeldungen sind in den meisten Fällen sehr deskriptiv was das Problem ist und gibt häufig auch gleich Lösungsvorschläge mit.

Besonders am Anfang kann der Umgang mit Referenzen herausfordernd sein. Deshalb empfiehlt sich zu Beginn nach diesen drei Regeln vorzugehen. Sie resultieren in Code, der zwar etwas weniger performant ist als er sein könnte, aber dafür kompiliert der Code. Da der Compiler viele Fehler und Edgecases bereits zur Compilezeit abfängt, die bei anderen Sprachen Runtime-Fehler auslösen würden, ist kompilierender Rust-Code in den meisten Fällen von solchen Fallstricken befreit.

1. **Keine Referenzen verwenden:** Übergib überall die Ownership und falls die Variabel später wieder benötigt wird, gib die Ownership wieder zurück
2. **Copy und Clone alle Daten:** Der Compiler meldet sich, wenn irgendwo eine Ownership-Regel verletzt wird. In der Fehlermeldung steht dann auch, dass man die `.clone()`-Funktion anwenden kann. Nimm das Angebot des Compilers an und dein Code kompiliert.
3. **Verwende nur Referenzen, wenn es der Compiler meldet:** Referenzen müssen ebenfalls nur dann verwendet werden, wenn es der Compiler vorgibt. Dies hilft einem mit dem Umgang von Referenzen und lässt das einem das Modell näherbringen.

Hat man diese drei Regeln genügend angewendet, sollte der Umgang mit Referenzen immer klarer werden und diese drei Regeln können hinter sich gelassen werden.

---

## 3. SLICES

Slices sind eine spezielle Art von Referenzen, die eine «Teil-Ansicht» eines Objekts erlauben, beispielsweise ein Teil eines Arrays oder Vektors. Sie erlauben es, auf manuelle Indizierung grösstenteils zu verzichten.

Slices verwenden die Schreibweise von **Ranges** mit `[<start>..end>]` gibt den ersten und den letzten Index an, letzterer ist nicht-inklusive. Mit einem `=` vor dem hinteren Index wird dieser aber auch inklusive. Wird eine Zahl nicht angegeben, ist es jeweils der Start oder das Ende des Elements.

```
let vec = vec![1, 2, 3, 4];
let middle = &vec[2..4];
let middle_too = &vec[2..=3];
let is_same = middle == middle_too;
println!("{middle:?} == {middle_too:?}: {is_same}");

let arr = ["Döner", "Burger", "Pizza"];
let full_slice = &arr[..];
let is_same = arr == full_slice;
println!("Equal? {is_same}");
```

Slices können auch mutable sein. Im Beispiel unten wird ein Array über zwei mutable Slices geändert. Die beiden gleichzeitigen mutable Referenzen sind möglich, da `x_full` nach der Definition von `x_last_two` nicht mehr verwendet wird, siehe Kapitel [«Lifetimes» \(Seite 25\)](#).

```
let mut x = [1, 2, 3];
let x_full = &mut x[..]; // first mutable slice takes whole array
x_full[1] = 7; // change second element in x
let x_last_two = &mut x[1..=2]; // second mutable slice takes second & third element
x_last_two[1] = 9; // change third element in x
println!("{x:?}"); // Output: [1, 7, 9]
```

Ranges können auch an anderen Orten eingesetzt werden, um beispielsweise schnell eine Liste von aufeinanderfolgenden Zahlen zu generieren

```
for i in (0..=5) {
    print!("{i}, "); // Output: 0, 1, 2, 3, 4, 5,
}
```

## 4. STRINGS

Strings sind in Rust immer UTF-8 codiert.

### 4.1. STRING VS. &STR

Rust hat mehrere Typen von Strings, was für Anfänger verwirrend sein kann. Die beiden hauptsächlichen Typen, welche man antrifft sind `String` und `&str` (String slice).

- **`String`**: Heap-alloziert, hat Ownership über den String (*Owned Type*)
- **`&str`**: Eine read-only Ansicht in einen String. Hat keine Ownership über den String (*Borrowed Type*)

String-Literale wie `let s = "Hello";` sind vom Typ `&str`<sup>2</sup> – sie sind im Binary mit einer festen Grösse hardcoded und können nicht direkt manipuliert werden.

Um sie modifizieren zu können, müssen sie auf den Heap gepackt werden. Das geschieht durch eine Umwandlung in einen `String`. Es gibt mehrere äquivalente Wege dafür, die Verwendung ist Geschmackssache:

```
let literal = "Convert me to String!";
let s1 = String::from(literal);
let s2 = literal.to_string();
let s3 = literal.to_owned();
```

Nimmt man einen Slice von einem `String`, erhält man einen `&str`. `String` kann implizit zu `&str` konvertiert werden, indem eine Referenz zu `String` mitgegeben wird. Im Beispiel unten wird mit einem `String Slice` das erste Wort in einem `String` zurückgegeben, indem nach einem Leerzeichen gesucht wird.

```
let string = String::from("Slice me");
let slice = &string[..];
let literal = "I'm a literal string!";

let first_str = first_word(slice);
let first_sli = first_word(&string);
let first_l = first_word(literal);
println!("{first_str}, {first_sli}, {first_l}");
// Output: Slice, Slice, I'm

fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }
    &s[..]
}
```

### Faustregel zur Verwendung von `String` und `&str`

In seinem Blog stellt der Autor von «The Book» Steve Klabnik<sup>3</sup> vier Levels von Regeln zur Verwendung dieser beiden Typen an. Level 2 stellt für Anfänger wohl die ideale Mitte dar:

*«Always use `String` in structs, and for functions, use `&str` for parameters and `String` types for return values.»*

— Steve Klabnik, «When should I use `String` vs `&str`»

In den meisten Fällen ist ein modifizierbarer `String` die beste Wahl. Parameter sind idealerweise `&str`, da `String` mit einem `&` automatisch zu `&str` konvertiert werden kann; die Funktion kann somit beide Typen annehmen. Wenn der übergebene `String` wieder zurückgegeben werden soll, kann als Rückgabetyt in den meisten Fällen wieder `String` verwendet werden.

### 4.2. STRING-FORMATIERUNG

Wie in den bisherigen Beispielen gesehen, können Text und Variablen mit dem `println!()` Makro ausgegeben werden. Alle Makros werden mit einem `!` nach dem Namen aufgerufen. Weitere Informationen finden sich im Kapitel «Macros» (Seite 35).

Das **`format!()` Makro** kann Strings wie `println!()` formatieren, gibt den `String` aber wieder zurück anstatt ihn auf der Konsole auszugeben. Es gibt eine grosse Anzahl an Formatierungsmöglichkeiten, darum hier nur die wichtigsten.

<sup>2</sup>Eigentlich sind sie vom Typ `&'static str`, aber die `static` Lifetime ist in diesem Fall nicht relevant

<sup>3</sup><https://steveklabnik.com/writing/when-should-i-use-string-vs-str/>

Die beiden Makros nehmen einen Format String und eine variable Anzahl von Parametern, welche in die {} Platzhalter eingefügt werden. Diese Platzhalter können direkt eine Variable enthalten, aber keine Expressions.

```
let name = "Rust";
let str1 = format!("I code in {name}");
let str2 = format!("I code in {}", name);
println!("str1 == str2? {}", str1 == str2); // Output: str1 == str2? true
println!("{}", str1.len()); // Compiler Error
println!("{}", str1.len()); // Compiler happy
```

### 4.3. AUF EINZELNE ZEICHEN ZUGREIFEN

Rust erlaubt keine String-Indizierung, um auf einzelne Zeichen zuzugreifen. String ist nur ein Wrapper um ein `Vec<u8>`, jedes Byte wird also separat abgespeichert. Das Problem ist, dass in UTF-8 Zeichen grösser als ein Byte sein können, deswegen kann bei der Indizierung ein Zeichen «abgeschnitten» werden.

String Slices, welche nur einen Teil eines Strings nehmen, sind zwar erlaubt, können aber bei multi-byte Charakteren zu Runtime Errors führen.

```
let two_byte_char_str = String::from("ä");
let slice = &two_byte_char_str[..1];
println!("{}", slice);
// Runtime Error: byte index 1 is not a char boundary; it is inside 'ä' (bytes 0..2) of `ä`
```

Rust bietet die `.chars()`-Funktion an, um trotzdem über einzelne Zeichen iterieren zu können.

```
let yummy = String::from("Chääs");
for char in yummy.chars() {
    println!("{}", char);
}
```

---

## 5. COMPOUND TYPES & COLLECTIONS

Neben den *Scalar Types*, die jeweils einen Wert repräsentieren (*i32*, *char*, *bool* etc.) gibt es in Rust auch *Compound Types*: Datentypen, welche andere Datentypen beinhalten können. Dazu gehören Arrays, Vectors und Tuples.

Eine gute Übersicht, wann welcher Collectiontyp verwendet werden soll bietet die [std::collection-Dokumentation](#).

### 5.1. ARRAYS

Arrays haben sehr ähnliche Funktionalitäten wie in den meisten anderen Programmiersprachen. Sie besitzen eine fixe Grösse, die zur Compilezeit festgelegt wird. Werden auf dem Stack alloziert. Die Typdefinition gibt den Datentyp und die Grösse in eckigen Klammern an. Mit einer Kurzschreibweise kann ein Array bei der Initialisierung mit demselben Wert gefüllt werden. Rust führt bei Indexzugriff automatisch einen Bounds-Check aus und beendet das Programm bei ungültigen Indizes.

```
let a1 = [1, 2, 3]; // infer size and data type => [i32; 3]
let a2: [f32; 3] = [1.1, 3.2, 4.2]; // explicitly specify data type and size
let a3 = [0; 4]; // creates a array with 4 zeroes
println!("{}", a3[2]); // Output: 0
```

### 5.2. VECTOR

Das Equivalent zu Listen in anderen Sprachen. Werden auf dem Heap alloziert und die Grösse kann dynamisch geändert werden. Können entweder leer initialisiert werden oder durch das `vec!` Makro direkt mit Werten befüllt werden.

```
let mut vec1 = Vec::new(); // Vec content type is inferred from .push()
let mut vec2 = vec![1, 2]; // Vec<i32> with 2 values
vec1.push(1); // Add value "1" to vec1
vec2.remove(1); // Remove value at index 1 from vec2
println!("Vecs equal? {}", vec1 == vec2); // Output: Vecs equal? true
```

Es kann entweder über eine Referenz + Index oder mit `get()` auf ein Element zugegriffen werden. Bei ungültigem Index stürzt die Referenz ab, während `get()` Error Handling ermöglicht.<sup>4</sup> Vektormodifikationen während eine Referenz gehalten wird, sind nicht möglich.

Über ein Vector kann auch mit einem `for`-Loop zugegriffen werden, dann erhält man eine Referenz des Wertes. Bei einem mutable Zugriff muss dabei der Wert dereferenziert werden.

```
let mut vec1 = vec![1, 2, 3];
let elem1 = &vec1[0]; // Reference access
let elem2 = vec1.get(1); // get() access
// Error: mut & immut borrow because of println
vec1.push(4);
println!("{elem1}");

for elem in &mut vec1 {
    *elem += 1;
}
for elem in &vec1 {
    println!("{elem}");
}
```

### 5.3. TUPLES

Tupel können mehrere Werte unterschiedlicher Typen zusammen speichern. Ihre Grösse und die enthaltenen Datentypen werden zur Compilezeit festgelegt. Der Zugriff erfolgt über `.<index>`. Sollten nur als «temporäre» Gruppierungen verwendet werden, für Klassen sollen Structs oder Tuple Structs verwendet werden.

```
let tuple: (String, i32) = ("Jannis".to_string(), 25);
let first = tuple.0;
let color: (u8, u8, u8) = (255, 128, 0);
let (red, green, blue) = color; // Destructuring syntax
```

### 5.4. HASHMAP

In anderen Sprachen bekannt als Dictionary. Besteht aus einer Liste von Key-Value-Pairs. Mit `insert()` können Werte hinzugefügt werden, mit `get()` erhält man eine `Option` mit einer Referenz auf den Wert. Siehe Kapitel «[Optionale Werte](#)» (Seite 12).

```
let mut scores = HashMap::new(); // Contained type gets inferred from first insert
scores.insert(String::from("Blue"), 10);
let team_name = String::from("Blue");
let score = scores.get(&team_name) // Returns a Option<i32>
    .copied() // Turns the Option<i32> into a Option<i32>
    .unwrap_or(0); // Gets the value if the key exists, 0 otherwise
scores.entry(&team_name) // Get the key/value pair
    .or_insert(0); // Insert the value only if the key doesn't exist
```

Eine Hashmap ist unsortiert, das heisst, die Keys folgen keiner bestimmten Reihenfolge. Ist eine Sortierung oder ein performanter Vergleich von Keys gewünscht, sollte eine `BTreeMap` verwendet werden.

---

## 6. ALGEBRAISCHE TYPEN

Das algebraische Typensystem in Rust erlaubt das Modellieren von Abhängigkeiten ohne die normalen OOP-Strukturen wie Vererbung, Polymorphismus und Enkapsulierung. Durch dieses Modell können invalide Zustände in einem System unrepräsentierbar gemacht werden.

Doch vorerst eine Einführung in die beiden grundlegenden Datenstrukturen des Typensystems: Structs und Enums.

### 6.1. STRUCTS

In Rust gibt es keine Klassen, sondern nur Structs. Sie funktionieren aber sehr ähnlich: Mit ihnen können verschiedene Daten zusammen gruppiert und Instanzen davon erstellt werden.

```
struct ColorRegularStruct {
    red: u8,
    green: u8,
    blue: u8,
    name: String,
}

let green = ColorRegularStruct {
    red: 0,
    green: 255,
    blue: 0,
    name: String::from("Green"),
};
```

---

<sup>4</sup>Gibt ein `Option<T>` zurück

Bei der Initialisierung kann der Feldname weggelassen werden, wenn eine Variabel denselben Namen hat.

```
let red = 128;
let color = ColorRegularStruct {
    red,          // instead of red: red
    green: 128,
    blue: 0,
    name: String::from("Orange")
}
```

Es gibt ebenfalls **Tuple Structs**, sie sind quasi «wiederverwendbare Tupel». Wie bei Tupeln haben die einzelnen Felder keine Namen. Sie sind geeignet, um wiederkehrende und zusammengehörende Werte wie beispielsweise Farbenwerte oder die Teile einer IPv4-Adresse zusammenzufassen und indirekt zu validieren.

```
struct ColorTupleStruct(u8, u8, u8, String);
struct IpAddrV4(u8, u8, u8, u8);
let red = ColorTupleStruct(255, 0, 0, String::from("Red"));
let home = IpAddrV4(127, 0, 0, 1);
```

### 6.1.1. Structs ausgeben: Die Display & Debug Traits

Dieses Kapitel ist ein kleiner Einschub über Traits. Für eine genauere Erklärung, siehe Kapitel «Traits» (Seite 21).

Soll ein Struct mit `println!()` ausgegeben werden, erhält man eine Fehlermeldung.

```
struct Rectangle { width: u32, height: u32 }
let r = Rectangle { width: 25, height: 10, };
println!("r is {r}"); // Error: `Rectangle` doesn't implement `std::fmt::Display`
```

Um ein Struct für die Ausgabe zu formatieren, wendet das `println!` Makro das Verhalten des `Display` Traits an (ähnlich wie `ToString()` in C#). Beim `Display` Trait gibt kein Standardverhalten, es muss bei jeder Klasse separat implementiert werden. Darauf wird genauer im Traits-Kapitel eingegangen.

Für die meisten Fälle genügt aber die **Debug-Ausgabe**, welche eine Standard-Implementation anbietet. Um diese für ein Struct zu aktivieren, muss dieser Trait für das entsprechende Struct **derive'd** und die Debug-Formatierung in `println!` mit «`:?`» aktiviert werden.

```
#[derive(Debug)] // Enables default implementation of Debug printing
struct Rectangle { width: u32, height: u32 }
let r = Rectangle { width: 25, height: 10, };
println!("r is {r:?}"); // Output: r is Rectangle { width: 25, height: 10 }
```

### 6.1.2. Methoden in Structs: impl-Blocks

Wie in anderen Sprachen können Structs auch Methoden beinhalten. Diese werden getrennt vom Struct in einem `impl`-Block implementiert. Jede Methode besitzt als ersten Parameter eine Referenz auf das eigene Struct mit `&self` oder `&mut self`. Felder können mit `self.<name>` aufgerufen werden.

```
struct Rectangle {
    width: u32,
    height: u32
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn can_contain(&self, other: Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}

let rect1 = Rectangle {
    width: 10,
    height: 20
};

let rect2 = Rectangle {
    width: 14,
    height: 7
};

let area = rect1.area();
let cont = rect1.can_contain(rect2);
```

Auch statische Methoden sind möglich, Rust nennt diese *associated functions*. Auf sie kann mit `<struct>::<func>` zugegriffen werden.

Damit können unter anderem Konstruktor-Funktionen implementiert werden; diese sind ganz normale Funktionen, welche `Self` (*gross geschrieben!*) zurückgeben. Per Konvention werden sie `new` genannt.

```
impl Rectangle {
    fn new(width: u32, height: u32) → Self {
        Self { width, height }
    }
    fn create_square(size: u32) → Self {
        Self {
            width: size,
            height: size,
        }
    }
}

let rect = Rectangle::new(7, 3);
println!("Rectangle: {rect:?}");
// Output (mit #[derive(Debug)]):
// Square: Rectangle { width: 7, height: 3 }

let square = Rectangle::create_square(15);
println!("Square: {square:?}");
// Output (mit #[derive(Debug)]):
// Square: Rectangle { width: 15, height: 15 }
```

## 6.2. ENUMS

Das Herz des algebraischen Typensystems bilden die Enums. Anders als in anderen Programmiersprachen können sie nicht nur mögliche nicht-überlappende Werte darstellen, sondern sie können auch Daten speichern. Dadurch können Hierarchien modelliert werden.

Im Beispiel unten haben wir ein Enum, welches verschiedene Arten von Nachrichten darstellt. Die Enum-Member speichern alle andere Datentypen ab. Diese Typen können ähnlich wie Structs initialisiert werden.

```
enum Message {
    Quit, // No Data
    Move { x: i32, y: i32 }, // Named fields
    Write(String), // Another type
    ChangeColor(i32, i32, i32), // Tuple
}

let messages = [ // Create array of messages
    Message::Move { x: 10, y: 30, },
    Message::Write(String::from("hello world")),
    Message::ChangeColor(200, 255, 255),
    Message::Quit,
];

fn process(msg: Message) {
    // ...
}

for msg in messages {
    process(msg);
}
```

### 6.2.1. Das match Statement

`match` ist das Rust-Equivalent zum Switch Statement. `match` zwingt aber den Nutzer, alle möglichen Zustände zu implementieren, ansonsten tritt ein Compilerfehler auf. Mit «`_`» kann jedoch ein Default case implementiert werden. Jeder Match-Arm endet in einem Komma. Mit geschweiften Klammern können mehrere Statements in einem Match-Arm verwendet werden.

```
let number = 13;
match number {
    1 ⇒ println!("One!"), // Match a single value
    2 | 3 | 5 | 7 | 11 ⇒ println!("Prime number"), // Match multiple values
    13..=19 ⇒ println!("A teen"), // Match an inclusive range
    _ ⇒ { println!("Ain't special"); return; }, // Default case
}
```

Damit kann jeder Zustand eines Enums matched werden. Wird einer vergessen, warnt der Compiler. Aus diesem Grund sollten `match` Statements `if`-Statements vorgezogen werden.

```
fn process(msg: Message) {
    match msg {
        Message::Move { width, height } ⇒ move_to(width, height),
        Message::Write(string) ⇒ write(string),
        Message::ChangeColor(red, green, blue) ⇒ change_color(red, green, blue),
        Message::Quit ⇒ quit(),
    }
}
```

Ein `match` kann aber nicht nur Konstanten überprüfen, sondern auch Pattern Matching betreiben – komplexe Datentypen können auf ihren Inhalt überprüft und ihr Inhalt an Variablen gebunden werden.

```
let tuple = (1, -2, 5);
match tuple {
  (0, y, z) => println!("x is 0, y: {y}, z: {z}"),
  (.., 5) => println!("z is 5, x & y don't matter"),
  (3, .., 4) => println!("x is 3 & z is 4"),
  _ => println!("Default Case")
}

struct User { track: bool, id: i64 }
let u = User { track: false, id: 69 };
match u {
  User { track: true }
    => println!("Track!"),
  User { track: false }
    => println!("Track anyway!") }
```

`match` kann ebenfalls zur Zuweisung einer Variabel verwendet werden.

```
let s1 = String::from("A string!")
let result = match s1.as_str() { // .as_str() allows matching String with String literals
  "Hello" => 0,
  "World" => 1,
  _ => 2,
}
```

`match` unterstützt ebenfalls zusätzliche `if`-Statements in Match Armen. Damit können auf verschachtelte Typen wie hier mit `Option::Some` (siehe «Optionale Werte» (Seite 12)) weitere Checks ausgeführt werden. Ein Nachteil ist aber, dass der Compiler nicht mehr prüfen kann, ob alle möglichen Zustände abgedeckt sind.

```
let num = Some(4);
match num {
  Some(x) if x % 2 == 0
    => println!("{x} is even"),
  Some(x) => println!("{x} is odd"),
  None => (),
}

let x = Some(5);
let y = 10;
match x {
  Some(24) => println!("x is 24"),
  Some(n) if n == y => println!("Matched, n = {n}"),
  _ => println!("Default case, x = {x:?}"),
}
```

### 6.3. BEISPIEL: SUPER MARIO POWER-UPS

Als Beispiel modellieren wir hier die Power-Ups aus dem SNES-Klassiker «Super Mario World» als Zustandsmaschine in Rust.<sup>5</sup> Zuerst erstellen wir zwei Enums, die Marios Zustand und den Übergang zwischen den Power-Ups modellieren:

```
enum State {
  Mario,
  SuperMario,
  FireMario,
  CapeMario
}

enum Transition {
  Feather,
  Flower,
  Mushroom
}
```

Nun implementieren wir ein `Player` Struct, welche den Zustand von Mario enthält. Zusätzlich implementieren wir für dieses Struct zwei Methoden: `new()`, welche als Konstruktor für `Player` den initialen state festlegt, und `collect_powerup()`, welche den Zustandswechsel modelliert.

Im `match` Statement erstellen wir ein Tupel, welches Marios Zustand und sein aufgesammeltes Power-Up enthält. Damit betreiben wir nun Pattern Matching: Wird ein Pilz gesammelt und er ist im Mario-Zustand, wird er zu SuperMario. Ist er allerdings in einem anderen Zustand, passiert bei einem Pilz nichts. Ähnlich funktionieren auch die anderen Power-Ups.

<sup>5</sup>aus dem Video <https://www.youtube.com/watch?v=z-0-bbc80JM>

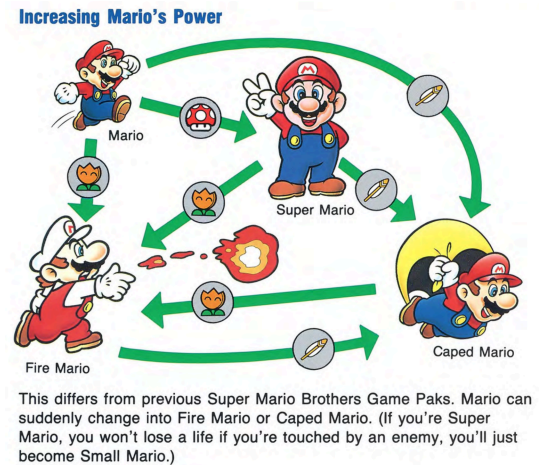
```

struct Player {
    state: State
}

impl Player {
    fn new() → Self {
        Self { state: Mario }
    }

    fn collect_powerup(&mut self, power_up: Transition) {
        match (&self.state, power_up) {
            (Mario, Mushroom) ⇒ self.state = SuperMario,
            (_, Flower) ⇒ self.state = FireMario,
            (_, Feather) ⇒ self.state = CapeMario,
            (_, Mushroom) ⇒ {} // Already has Mushroom
        }
    }
}

```



Weil der Compiler überprüft, ob alle möglichen Zustände im `match` Statement abgedeckt sind, gibt es keine Edge Cases. Damit entfällt auch das Schreiben von Tests, da der Compiler dies schon für uns überprüft.

```

let mut player = Player::new();
player.collect(Transition::Mushroom);
player.collect(Transition::Flower);
player.collect(Transition::Feather);
player.collect(Transition::Mushroom);
player.collect(Transition::Mushroom);
assert!(player.state == CapeMario);

```

## 7. OPTIONALE WERTE

In Rust gibt es kein `null`. Soll das Vorhandensein eines Wertes modelliert werden, kann das `Option<T>` Enum verwendet werden. Vergleichbar mit Optionalen Typen wie `int?` in C#.

```

enum Option<T> {
    None,
    Some(T),
}

```

- **None**: Es ist kein Wert vorhanden, nichts wird mitgegeben
- **Some(T)**: Ein Wert ist vorhanden und wird mitgegeben

`Option` wird häufig mit dem `match` Statement geprüft und daraufhin ausgewertet. Da `Option` im Prelude der Standard Library ist, kann der `Option::` Prefix vor `None` und `Some` weggelassen werden.

```

fn divide(dividend: f32, divisor: f32) → Option<f32> {
    if divisor == 0.0 {
        None
    } else {
        Some(dividend / divisor)
    }
}

let result_option = divide(5.0, 2.0);

match result_option {
    Some(result) ⇒ println!("{result}"),
    None ⇒ println!("No result!")
}

```

### 7.1. SIMPLERE CHECKS MIT IF-LET UND LET-ELSE

In vielen Fällen ist ein `match`-Statement überflüssig, weil beispielsweise nur Code ausgeführt werden soll, wenn ein Wert vorhanden ist. `if-let` funktioniert ähnlich wie `match`, mit dem Unterschied, dass nur ein Pattern matched wird und alle anderen ignoriert.

Der Beispielcode sagt aus: Wenn `max_value_option` vom Typ `Some` ist, speichere dessen Inhalt in `max_value` und führe den Code in den geschweiften Klammern aus. Der Wert wird also aus `Option` herausgenommen, wenn er existiert.

```
let max_value_option = Some(100);
if let Some(max_value) = max_value_option {
    println!("Max value is set at {max_value}");
}
```

Bei `if-let` ist die neue Variabel aber nur innerhalb der geschweiften Klammern gültig. Rust hat deswegen `let-else`, welches den Code in den geschweiften Klammern ausführt, wenn das Pattern nicht matched.

Der Code unten sagt also aus: Wenn `max_value_option` vom Typ `Some` ist, speichere dessen Inhalt in `max_value`. Wenn nicht, führe den Code in den geschweiften Klammern aus.

```
let max_value_option = Some(100);
let Some(max_value) = max_value_option else {
    println!("Max value missing!");
    return;
};
println!("Max value {max_value} available here!");
```

---

## 8. ERROR HANDLING

Rust bietet umfangreiches Error Handling, welches man sich einfacher oder komplexer machen kann. Rust hat keine Exceptions wie beispielsweise C#, sondern Errors, welche den Programmflow nicht direkt unterbrechen.

### 8.1. PANIC!

Wird das `panic!()` Makro aufgerufen, beendet sich das Programm und eine Meldung wird in der Konsole angezeigt. Die simpelste Art von Error Handling. Sollte nur von Funktionen aufgerufen werden, bei denen der Zustand durch einen Fehler unkorrigierbar wird.

```
fn main() {
    panic!("at the Disco");
}

thread 'main' panicked at src/main.rs:2:5:
at the Disco
note: run with `RUST_BACKTRACE=1` environment
variable to display a backtrace
```

```
fn main() {
    let v = vec![1, 2, 3];
    v[99]; // calls panic!() implicitly
}

thread 'main' panicked at src/main.rs:3:6:
index out of bounds: the len is 3 but the index
is 99
```

### 8.2. RESULT<T, E>

Das `Result<T, E>` Enum ist dem `Option<T>` sehr ähnlich. Es dient zur Übergabe eines Wertes oder eines Fehlers.

|                                                               |                                                                                                                                                                                                                    |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>enum Result&lt;T, E&gt; {     Ok(T),     Err(E), }</pre> | <ul style="list-style-type: none"><li>– <code>Ok(T)</code>: Die Operation war erfolgreich, der Wert wird mitgegeben</li><li>– <code>Err(E)</code>: Die Operation schlug fehl, ein Fehler wird mitgegeben</li></ul> |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Gibt eine Funktion ein `Result` zurück, kann dies wie bei `Option` mit `match` behandelt werden.

```
let file_result = File::open("hello.txt");
let file = match file_result {
    Ok(file) => file,
    Err(error) => panic!("Problem opening the file: {error:?}"),
};
```

### 8.2.1. Eigene Error-Typen

Der Wert von E kann jeder mögliche Typ sein. Für viele Situationen stellt Rusts Standard Library bereits eigene Errortypen bereit, man kann aber auch eigene Errortypen erstellen. Idealerweise wird dafür ein Enum verwendet, da immer nur ein Error zurückgegeben werden kann.

Im Beispiel unten haben wir ein Struct, welches positive Integer grösser als Null abspeichern soll. Wir implementieren eine new Methode, welche ein neues Objekt erstellt. Das Konstruieren kann aus zwei Gründen fehlschlagen: Die Zahl ist negativ oder 0. Dazu erstellen wir ein CreationError Enum, die diese Fehler abbildet.

```
struct PositiveNonzeroInteger(u64);
impl PositiveNonzeroInteger {
    fn new(value: i64) → Result<Self, CreationError> {
        match value {
            x if x < 0 ⇒ Err(CreationError::Negative),
            0 ⇒ Err(CreationError::Zero),
            x ⇒ Ok(PositiveNonzeroInteger(x as u64)),
        } } }

enum CreationError {
    Negative,
    Zero
}
```

### 8.2.2. Result vs. Option

Semantisch gesehen sagt Option eines von zwei Dingen aus:

- Das Nicht-Vorhandensein eines Wertes wird erwartet, repräsentiert also normales Verhalten
- Der Grund, warum None zurückgegeben wird, ist eindeutig

**Beispiel:** HashMap::get() gibt einen Wert zurück, wenn der Key vorhanden ist. Damit gibt es genau einen eindeutigen Grund, warum die Operation fehlschlagen kann – Fehlen des Keys. Die Funktion returnt also Option::None.

Result sagt das Gegenteil davon aus:

- Das Nicht-Vorhandensein eines Wertes wird nicht erwartet
- Es gibt mehrere Gründe, warum die Operation fehlschlagen kann

**Beispiel:** Das Parsen von einem String in einen Integer kann mehrere Ursachen haben: Der String ist leer, enthält keine Ziffern oder die Zahl ist zu gross für den gewählten Integer. Die Funktion returnt bei Fehler einen Result::Err(ParseIntError).

### 8.2.3. Konvertierung Result ⇔ Option

Für die Konvertierung von Result und Option ineinander werden entsprechende Funktionen bereitgestellt.

- **Result::ok():** Konvertiert ein Ok(T) in ein Some(T) bzw. ein Err(E) in ein None
- **Option::ok\_or(e):** Konvertiert ein Some(T) in ein Ok(T) bzw. konstruiert ein Err(E) mit dem Parameter e
- **Option::ok\_or\_else(||):** Nimmt anstatt eines statischen Werts ein Closure entgegen, welches den Error erstellt

## 8.3. KÜRZERES ERROR HANDLING MIT UNWRAP/EXPECT

Häufig ist ein match-Konstrukt zu verbos, da man nur an einem erfolgreichen Resultat interessiert ist. Dasselbe Verhalten wie im initialen Beispiel kann mit den Funktionen unwrap() und expect() erreicht werden. Beide rufen panic!() auf, wenn ein **Result vom Typ Err** oder **Option vom Typ None** ist. Bei expect kann noch eine eigene Nachricht mitgegeben werden, wieso dieser Aufruf nie fehlschlagen sollte. Laut Styleguide sollten diese Nachrichten den Soll-Zustand mit «Should...» beschreiben.<sup>6</sup>

```
let file1 = File::open("hello.txt").unwrap(); // panics if file can't be opened.
let file2 = File::open("bye.txt").expect("File should exist in the project"); // panics with msg
```

<sup>6</sup><https://doc.rust-lang.org/std/error/index.html#common-message-styles>

Es gibt weitere Variationen von `unwrap`, die anstatt `panic!()` etwas anderes ausführen. `unwrap_or()` kann statische Werte als «Default» setzen, `unwrap_or_else()` kann ein Closure (siehe «Closures» (Seite 17)) ausführen.

```
let default = 7;
let array = [1, 2];
let result = array.get(3).unwrap_or(&default);
println!("{result}"); // Output: 7

let file =
File::open("no.txt").unwrap_or_else(|err|
{
    println!("Problem opening file: {err}");
    process::exit(1);
});
```

`unwrap()` ist ideal für Prototyp-Code, bei welchem das Error Handling wenig relevant ist. In Production-Code sollte es aber nicht auftauchen. Das Verwenden von `unwrap()` signalisiert aber klar, dass sich hier eine «unsichere» Codestelle befindet und hier sich um richtiges Errorhandling gekümmert werden sollte.

#### 8.4. ERROR PROPAGATION: DER ? (TRY) OPERATOR

Oft ist es erwünscht, ein Fehler in einer Funktion an den Caller weiterzureichen. Dazu wird als Rückgabetypp `Result` mit dem entsprechenden Error verwendet. Nun kann der Aufrufer selbst bestimmen, wie mit dem Fehler umgegangen werden soll.

```
fn read_file() → Result<String, io::Error> {
    let file_result = File::open("hello.txt");
    let mut file_handle = match file_result {
        Ok(file) ⇒ file,
        Err(e) ⇒ return Err(e),
    };

    let mut content = String::new();
    match file_handle.read_to_string(&mut content) {
        Ok(_) ⇒ Ok(content),
        Err(e) ⇒ Err(e),
    }
}
```

Auch dieser Code ist ziemlich verbos. Wenn in einem `match`-Statement nur der Error zurückgegeben werden soll, kann der **Try Operator ?** hinter dem `Result` verwendet werden, um den Code kürzer zu gestalten. Dazu muss die Funktion, in welcher er verwendet wird, `Result` returnen.

```
fn read_file() → Result<String, io::Error> {
    let file = File::open("hello.txt"?);
    let mut content = String::new();
    file.read_to_string(&mut content)?;
    Ok(content)
}
```

Der try Operator kann auch verkettet werden, um den Code noch kürzer zu gestalten.

```
fn read_file() → Result<String, io::Error> {
    let mut content = String::new();
    File::open("hello.txt"?).read_to_string(&mut content)?;
    Ok(content)
}
```

## 8.5. BEISPIEL: GRANDPARENT IN EINEM TREE

In diesem Beispiel haben wir ein Struct `Node`, welches einen Node in einem Tree repräsentiert. `Node` implementiert zwei Funktionen: `parent()`, die eine Referenz zum überliegenden Node zurückgibt und `grandparent()`, was den Parent des Parent zurückgibt.

Die `parent()` Methode kann nur auf eine Art fehlschlagen: Indem der Node keinen Parent hat, also der Root Node ist. Deswegen returnt dieser `Option`.

`grandparent()` ruft zwei Mal `parent()` auf, kann also auf zwei Arten fehlschlagen: Der aktuelle oder der Parent Node sind der Root Node. Möchte man diese Information dem Caller übergeben, muss der Return Type ein Error sein.

```
struct Node { /* ... */ };
impl Node {
    fn parent(&self) → Option<&Node> { /* ... */ }
    fn grandparent(n: &Node) → Result<&Node, GrandpErr> {
        Ok(
            n.parent().ok_or(GrandpErr::NoParent)?
            .parent().ok_or(GrandpErr::NoGrandparent)?
        )
    }
}
```

```
enum GrandpErr {
    NoParent,
    NoGrandparent
}
```

`grandparent()` wandelt die `Options` von den beiden `parent()` Calls in `Results` um. Returnt `parent()` `None`, wird ein `Err` mit dem entsprechenden `GrandpErr` erstellt und mit dem `try` operator zum Caller returnt.

---

## 9. PROJEKTSTRUKTUR

Rust hat mehrere Features zur Strukturierung des Codes und Projekte.

- **Paths:** Der volle Name eines Objekts
- **Modules:** Kontrollieren die Organisation, Scope und Privacy von Paths
- **Crates:** Sammlung von Modulen, die ein Programm oder eine Library produziert
- **Packages:** Sammlung von Crates, die bestimmte Funktionalitäten bereitstellt

### 9.1. CRATES

Crates sind die kleinsten Einheiten, die der Compiler berücksichtigt. Auch eine einzige Quellcodedatei gilt bereits als Crate. Sie können zwei Formen auftreten.

- **Binary Crate:** Kompiliert zu einer Anwendung, hat eine `main()`-Funktion
- **Library Crate:** Kompiliert zu einer Library, die von anderen Crates verwendet werden kann

Die **Crate Root** ist die Quellcodedatei, von welcher der Compiler anfängt, den Code zu kompilieren. Bei Binary Crates ist das standardmässig `src/main.rs`, bei Library Crates `src/lib.rs`

### 9.2. PACKAGE

Sammlung von mehreren Crates. Hat eine `Cargo.toml`-Datei, die Build-Instruktionen beinhaltet. Ein Package kann beliebig viele Binary Crates beinhalten, aber nur maximal eine Library Crate.

Ein Package wird mit `cargo new` erstellt. Das kreiert automatisch die Projektstruktur, wie `Cargo.toml`, `src/main.rs` und ein Git Repo.

Packages, die eine Binary Crate beinhalten, sollten so viel Code wie möglich in eine Library Crate auslagern. Damit wird die Binary Crate zum Benutzer der Library Crate, als wäre sie eine komplett externe Crate; sie kann nur die Public API verwenden, was zu einem besseren API Design führt.

### 9.3. MODULES

In der Crate Root können neue Module deklariert werden. Mit dem Statement `mod garden`; wird ein neues Modul namens `garden` erstellt. Der Compiler sucht nun nacheinander an drei Orten nach diesem Modul:

1. Inline in derselben Datei innerhalb von `mod garden {}`
2. In der Datei `src/garden.rs`
3. In der Datei `src/garden/mod.rs`

Wird der Code in eine andere Datei ausgelagert, können ebenfalls noch Submodule deklariert werden. (z.B. mit `mod vegetables`; in `src/garden.rs`). Diese werden auf die selbe Art wie reguläre Module gesucht.

### 9.4. VISIBILITY

Der Code innerhalb eines Modules ist *standardmässig private*. Um das Modul `public` zu machen und damit von anderen Crates verwendet werden kann, muss es als `pub mod` deklariert sein. Damit wird aber noch kein Zugriff auf die Structs, Enums und Funktionen innerhalb eines `public` Moduls gewährt, diese müssen noch einzeln als `pub` deklariert werden. Bei Structs müssen Felder ebenfalls einzeln als `pub` markiert werden, bei Enums aber nicht.

Zwischen `public` und `private` gibt es noch weitere Abstufungen:

- `pub(super)`: Das Element ist für das Parent-Modul sichtbar.
- `pub(crate)`: Das Element ist innerhalb der Crate sichtbar

### 9.5. PATHS

Damit auf ein Element in einem Modul zugegriffen werden kann, muss ein absoluter oder relativer Pfad verwendet werden. Modulnamen werden durch `« :: »` abgegrenzt. Damit nicht immer der vollständige Pfad geschrieben werden muss, kann auch mit dem `use` keyword das Modul in den Scope gebracht werden.

Der Rust Styleguide gibt vor, bei Funktionen das Parentmodule in das `use` Statement zu nehmen, während bei Structs und Enums der ganze Pfad spezifiziert wird.

```
let c1 = crate::garden::vegetables::carrot::new();    crate::garden::vegetables::print_all();
use crate::garden::vegetables::carrot;                use crate::garden::vegetables;
let c2 = carrot::new();                                vegetables::print_all();
```

Es gibt zwei spezielle Keywords in einem `use` Statement, die am Anfang eines Pfades stehen können

- `crate`: Absoluter Pfad der aktuellen Crate
- `super`: Namespace des überliegenden Moduls

Pfade können auch mit dem `as` Keyword umbenannt werden, um Namenskonflikte vorzubeugen.

```
use std::fmt::Result;
use std::io::Result as IoResult;
```

Sind mehrere Elementen im selben Modul oder derselben Crate, können sie auch mit `{}` genested werden.

```
use std::{io, env};
```

---

## 10. CLOSURES

Closures sind anonyme Funktionen, aus anderen Sprachen bekannt als Lambdas. Ein Closure wird mit `||` definiert, eventuelle Parameter, sogenannte *Captures*, befinden sich zwischen den beiden `|`. Sie können in Variablen gespeichert werden.

```
let preference1 = Some(Color::Blue);
let preference2 = None;
let result1 = preference1.unwrap_or_else(|| get_random_color()); // Output: Color::Blue
let result2 = preference2.unwrap_or_else(|| get_random_color()); // Output: Random color
let is_equal = |a, b| a == b;
let same_colors = is_equal(result1, result2);
```

Closures können auch auf äussere Variablen zugreifen. Wird nur lesend auf eine Variabel zugegriffen, wird ein immutable borrow durchgeführt, ansonsten ein mutable borrow. Bei Mutable Borrowings wird die Variabel bereits bei der Definition des Closure borrowed, ein Zugriff auf diese Daten ist dann bis zum letzten Aufruf dieses Closures nicht mehr möglich.

```
let list = vec![1, 2, 3];
let immut_closure = || println!("{list:?}");
println!("Before closure: {list:?}");
immut_closure();
println!("After closure: {list:?}");

let mut list = vec![1, 2, 3];
let mut mut_closure = |x| list.push(x);
// println!("{list:?}") not OK => mut borrow
mut_closure(4);
println!("After closure: {list:?}"); // OK again
```

Die Typen der Parameter und des Rückgabewertes in einem Closure können meistens vom Compiler erkannt werden. Beim ersten Aufruf des Closures werden diese festgelegt. Es ist also nicht möglich, ein Closure mehrmals mit unterschiedlichen Typen aufzurufen.

```
let closure = |x| x; // Returns itself
let i = closure(7);
let f = closure(9.5); // Compiler error: mismatched types. expected because the closure was
// earlier called with an argument of type `integer`
```

## 10.1. FUNKTIONSTYPEN

Closures werden in drei Kategorien eingeteilt, je nach dem wie sie mit ihrer Umgebung interagieren

- **Fn**: Ein Closure, welches Werte nur lesend captured oder gar nichts captured. Sie verändern an ihrer Umgebung nichts, können also mehrmals aufgerufen werden
- **FnMut**: Ein Closure, welches gecapturte Werte verändert. Diese können mehrmals aufgerufen werden, können aber ein anderes Resultat zurückgeben.
- **FnOnce**: Ein Closure, welches Ownership über ein Capture übernimmt. Das Capture wird damit konsumiert, das Closure kann also nur einmal aufgerufen werden

```
fn create_typed_closures() -> (
    impl Fn(f64, f64) -> f64,
    impl FnMut(&mut f64, f64),
    impl FnOnce(String) -> String
) {
    let total = |price, tax_rate| { price + price * tax_rate };
    let apply_discount = |price: &mut f64, discount| { *price = *price - discount };
    let checkout_cart = |cart| { format!("Checkout complete: {cart}") };
    (calculate_total, apply_discount, checkout_cart)
}

fn main() {
    let (calculate_total, mut apply_discount, checkout_cart) = create_typed_closures();
    assert_eq!(calculate_total(100.0, 0.2), 120.0);

    let mut total_price = 120.0;
    apply_discount(&mut total_price, 20.0);
    assert_eq!(total_price, 100.0);

    let cart_details = String::from("Items: Apple, Banana, Orange");
    assert_eq!(checkout_cart(cart_details), "Checkout complete: Items: Apple, Banana, Orange");
}
```

---

## 11. ITERATOREN

Ein Iterator kann über eine Collection loopen und für jedes Element Aktionen ausführen, vereinfacht gesagt ein `for-each` loop. Sie haben eine `next()`-Methode, die das nächste Element aufruft – der Iterator wird dadurch **konsumiert**. Ein Iterator gibt den Wert in einem `Some` zurück und `returnt None`, wenn auf dem letzten Element `next()` aufgerufen wird.

```
let v1 = vec![1, 2, 3];
let mut v1_iter = v1.iter();

assert_eq!(v1_iter.next(), Some(&1));
assert_eq!(v1_iter.next(), Some(&2));
assert_eq!(v1_iter.next(), Some(&3));
assert_eq!(v1_iter.next(), None);
```

`next()` muss häufig nicht manuell aufgerufen werden, die Funktionen, welche auf dem Iterator ausgeführt werden, erledigen dies meist für einen.

Es gibt drei verschiedene Arten von Iteratoren:

- **`iter()`**: Iteriert über immutable Referenzen
- **`iter_mut()`**: Iteriert über mutable Referenzen
- **`into_iter()`**: Erhält Ownership über die Collection und iteriert über Owned Values. Häufigster Einsatzzweck ist, um Objekte von einer Collection an einen anderen Ort zu verschieben

Ein Iterator an sich bewirkt noch nichts, es müssen zuerst **Adapter** darauf ausgeführt werden. Es gibt zwei Arten von Adaptern:

- **`Consuming Adapter`**: Konsumieren den Iterator, indem sie alle Elemente im Iterator benutzen. Sind **`eager`**, werden also sofort ausgeführt.
- **`Iterator Adapter`**: Verändern den bestehenden Iterator und geben einen neuen zurück. Konsumieren den Iterator nicht. Sind **`lazy`**, werden erst beim Konsumieren ausgeführt.

```
let vec = vec![4, 6, 5];
let sum = vec
    .iter() // Create Iterator
    .sum(); // Sum up values in the iterator

println!("{sum}"); // Output: 15

let vec = vec![4, 6, 5];
// collect() needs type annotation
// but inner type can be inferred
let vec_new: Vec<_> = vec.iter()
    .map(|x| x + 1) // Add 1 to each value
    .collect();    // Transform into collection
println!("{vec_new:?}"); // Output: [5, 7, 6]
```

`sum()` und `collect()` sind also Consuming Adapter, während `map()` ein Iterator Adapter ist.

### 11.1. HÄUFIG VERWENDETE ADAPTER

Iteratoren unterstützen viele aus funktionellen Sprachen bekannte Funktionen. Eine vollständige Liste befindet sich in der [Iterator-Trait Dokumentation](#).

**`for_each()`**: Wendet eine Funktion auf alle Elemente an

```
let arr = [6, 11, 7, 16];
arr.iter().enumerate().for_each(|(e, i)| println!("{i}: {e}"));
```

**`map()`**: Wendet eine Funktion auf jedes Element an und gibt den veränderten Iterator zurück

```
let vec_double: Vec<_> = arr.iter().map(|x| x * 2).collect();
```

**`enumerate()`**: Gibt das Element und dessen Index zurück

```
for (elem, index) in arr.iter().enumerate() {
    println!("{index}: {elem}");
}
```

**collect():** Nimmt alle Elemente und packt sie in einen Container. Der Container muss meist noch spezifiziert werden, entweder durch Typannotation der Variabel oder durch die **Turbofish Syntax** `:: <>`. Der Inhalt des Containers kann inferred werden und wird darum häufig mit einem Underscore angegeben.

```
let type_annot: Vec<_> = arr.iter.map(|x| x + 1).collect();
let turbofish = arr.iter.map(|x| x + 1).collect::
```

**filter():** Nimmt alle Elemente, auf die das Prädikat zutrifft. Closure-Parameter ist eine Referenz, muss darum dereferenziert werden.

```
let filtered: Vec<_> = arr.iter().filter(|x| *x % 2 == 0).collect();
```

**flatten():** Entfernt ein Level von Verschachtelung

```
let data = vec![vec![1, 2, 3, 4], vec![5, 6]];
let flattened: Vec<_> = data.into_iter().flatten().collect();
assert_eq!(flattened, [1, 2, 3, 4, 5, 6]);
```

**inspect():** Debugging-Funktion, um Elemente in einer Iterator-Pipeline zu loggen

```
let sum = a.iter()
    .cloned()
    .inspect(|x| println!("about to filter: {x}"))
    .filter(|x| x % 2 == 0)
    .inspect(|x| println!("made it through filter: {x}"))
    .fold(0, |sum, i| sum + i);
```

---

## 12. GENERICS

Wie bereits an einigen Datenstrukturen gesehen, erlaubt Rust auch Generics. Ein generischer Typ wird meist mit T gekennzeichnet. Zur Compilezeit wird neuer Code generiert, der T mit dem richtigen Typen ersetzt.

### 12.1. IN STRUCTS, ENUMS UND METHODEN

Am Beispiel des `Result<T, E>` Enums haben wir Generics schon gesehen. Sie funktionieren für Structs gleich.

```
enum Result<T, E> { // Two different Types
    Ok(T),
    Err(E),
}

struct Point<T> { // The same type
    x: T,
    y: T,
}
```

Jetzt können sowohl generische als auch spezifische Methoden auf einen Typ implementiert werden. Bei generischen Methoden muss bei `impl` ebenfalls die generischen Typen angegeben werden.

```
impl<T> Point<T> { // generic method
    fn get_x(&self) -> &T {
        &self.x
    }
}

impl Point<f32> { // specific method
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}
```

Es ist nicht möglich, generische und spezifische Methoden mit demselben Namen zu haben.

## 12.2. IN FUNKTIONEN

Generische Parameter haben keine Funktionalitäten. Damit sinnvoll mit Generics in Funktionen gearbeitet werden kann, müssen Generics eingeschränkt werden. Dazu wird nach dem Funktionsnamen die entsprechende Typ-Einschränkung notiert. Keine Sorge, der Compiler sagt in den meisten Fällen, welche Einschränkungen nötig sind.

```
// Does not compile
// Gets the largest item in a list
fn largest<T>(list: &[T]) → &T {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}
```

```
error[E0369]: binary operation `>` cannot be applied to
type `&T`
   → src/main.rs:5:17
   |
5 |         if item > largest {
   |             ---- ^ ----- &T
   |             |
   |             &T
   |
help: consider restricting type parameter `T` with trait
`PartialOrd`
   1 | fn largest<T: std::cmp::PartialOrd>(list: &[T]) → &T {
   |
```

## 13. TRAITS

Traits sind am besten vergleichbar mit Interfaces aus anderen Programmiersprachen. Mit ihnen kann bestimmtes Verhalten über mehrere Typen hinweg implementiert werden, indem die Funktionen des Traits zu definieren.

### 13.1. TRAITS AUS DER STANDARD LIBRARY

Die Rust Standard Library bietet viele nützliche Traits für eigene Typen an. Werden diese für eigene Typen implementiert, können sie die entsprechenden Funktionalitäten nutzen. Einige davon können automatisch mit einer *Derive* Annotation implementiert werden. Für eine manuelle Implementation steht in der Rust Dokumentation beim jeweiligen Trait unter «Required Methods» die Signatur der Funktionen, welche für diesen Trait geschrieben werden müssen.

| Trait      | Funktionalität                                                                                                                                                                                     | Derivable |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Display    | Ausgabe von Typen in Format Strings ( <i>println!()</i> , <i>format!()</i> ) in eigenem Format                                                                                                     | ✗         |
| Debug      | Ausgabe von Typen in Format Strings als Debug Output mit <i>:? (Name &amp; kommaseparierte Felder mit Inhalt)</i>                                                                                  | ✓         |
| PartialEq  | Implementation der <i>=/≠</i> Operatoren, wenn Objekte eines Typen equivalent sind (z.B. <i>case-insensitive Stringvergleich</i> ) oder nicht alle Elemente vergleichbar sind ( <i>NaN ≠ NaN</i> ) | ✓         |
| Eq         | Implementation der <i>=/≠</i> Operatoren, wenn Objekte eines Typen exakt gleich sind                                                                                                               | ✓         |
| PartialOrd | Implementation der <i>&lt;/&gt;/≤/≥</i> Operatoren, wenn nicht alle Werte vergleichbar sind                                                                                                        | ✓         |
| Ord        | Implementation der <i>&lt;/&gt;/≤/≥</i> Operatoren, wenn alle Werte vergleichbar sind                                                                                                              | ✓         |
| Default    | Setzen von Standardwerten in Structs                                                                                                                                                               | ✓         |
| From       | Konvertiere Parameter in diesen Typen (z.B. <i>String::from()</i> )                                                                                                                                | ✗         |
| Into       | Konvertiere diesen Typen in den (meist inferred) Inputtyp (z.B. <i>let s: String = "a".into()</i> )                                                                                                | ✗         |
| Clone      | Markiert einen Typ als Cloneable ( <i>Enthält Daten auf dem Heap</i> ). Alle Felder müssen Clone implementieren                                                                                    | ✓         |
| Copy       | Markiert einen Typ als Moveable ( <i>Alle Daten auf dem Stack</i> ). Alle Felder müssen Copy implementieren                                                                                        | ✓         |
| FromStr    | Kreiere eine neue Instanz eines Typen aus einem String                                                                                                                                             | ✗         |

### 13.1.1. Beispiel: Display Trait implementieren

Öffnet man die [Rust Dokumentation](#) für den `Display Trait` steht unter «Required Methods», was implementiert werden muss.

```
fn fmt(&self, f: &mut Formatter<'_>) → Result<(), Error>
```

Die Signatur scheint auf den ersten Blick etwas fremd, aber die Implementation ist nicht sehr aufwändig. Der Formatter `f` muss zusammen mit dem Format String, den wir spezifizieren, dem `write!()` Makro übergeben werden.

```
use std::fmt::{Display, Formatter, Result};
```

```
struct Point { x: i32, y: i32 }
```

```
impl Display for Point {
    fn fmt(&self, f: &mut Formatter<'_>) → Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}
```

```
fn main() {
    let p = Point { x: 3, y: -6 };
    println!("Point at {p}"); // Output: Point at (3, 6)
}
```

Das nächste Beispiel ist etwas komplexer: Eine Matrix wird durch verschachtelte Vektoren repräsentiert. Wir wollen deren Inhalt nun als mathematische Matrix ausgeben.

```
use std::fmt::{Display, Formatter, Result};
```

```
struct Matrix(Vec<Vec<i32>>);
```

```
impl Display for Matrix {
    fn fmt(&self, f: &mut Formatter<'_>) → Result {
        for (i, row) in self.0.iter().enumerate() { // Iterate over the outer Vec
            write!(f, "[")?; // Write opening bracket

            for (i, col) in row.iter().enumerate() { // Iterate over the inner Vec
                write!(f, "{col}")?; // Write the matrix values
                if i < row.len() - 1 {
                    write!(f, ", ")?; // Write a comma between the values
                }

                write!(f, "]")?; // Write closing bracket
                if i < self.0.len() - 1 {
                    write!(f, ",\n")?; // Write a comma & newline between the rows
                }
            }
            Ok(()) // Return OK when no errors occurred in write!
        }
    }
}

fn main() {
    let m = Matrix( // Output:
        vec![ // Matrix:
            vec![1, 2, 3], // [1, 2, 3],
            vec![4, 5, 6], // [4, 5, 6],
            vec![7, 8, 9] // [7, 8, 9]
        ]
    );
    println!("Matrix:\n{m}");
}
```

## 13.2. EIGENE TRAITS

Ein Trait wird in einem trait-Block definiert und enthält alle Methodensignaturen, welche auf Typen implementiert werden sollen.

```
trait Drinkable {  
    fn drink(&self) → Effect;  
}
```

Jeder Typ, der diesen Trait implementieren will, kann dies in einem impl-for-Block tun. Da alles Methoden sind, ist der erste Parameter &self.

```
struct Beer {  
    brand: String  
}  
struct Acid {}  
#[derive(Debug)]  
enum Effect {  
    None,  
    Alcohol,  
    Death,  
}  
  
impl Drinkable for Beer {  
    fn drink(&self) → Effect {  
        println!("Drinking {} beer!", self.brand);  
        Effect::Alcohol  
    }  
}  
  
impl Drinkable for Acid {  
    fn drink(&self) → Effect {  
        println!("Died from Acid Poisioning");  
        Effect::Death  
    }  
}
```

Der Trait muss in anderen Source Files in den Scope gebracht werden.

```
use mycrate::{Beer, Acid, Effect, Drinkable};  
fn main() {  
    let beer = Beer { brand: "Quöllli".to_string() };  
    let acid = Acid {};  
    let rb = beer.drink();  
    let ra = acid.drink();  
    println!("beer.drink: {rb:?}, acid.drink: {ra:?}");  
}
```

// Output:  
// Drinking Quöllli beer!  
// Died from Acid Poisioning!  
// beer.drink: Alcohol, acid.drink: Death

## 13.3. DEFAULT IMPLEMENTATION VON TRAITS

Es kann auch eine Default-Implementation angeboten werden, die von einzelnen Klassen überschrieben werden kann.

```
trait Drinkable {  
    fn drink(&self);  
  
    fn spill(&self) {  
        Effect::None  
    }  
}  
  
impl Drinkable for Acid {  
    fn drink(&self) → Effect {  
        Effect::Death  
    }  
    fn spill(&self) {  
        Effect::Death  
    }  
}
```

Die nicht implementierte spill()-Funktion bei Beer gibt nun den Wert der Standartimplementation aus.

```
let beer = Beer { brand: "Quöllli".to_string() };  
let acid = Acid {};  
let rb = beer.spill();  
let ra = acid.spill();  
println!("Spilling Beer: {rb:?}, Spilling Acid: {ra:?}");  
// Output: Spilling Beer: None, Spilling Acid: Death
```

### 13.4. TRAITS ALS PARAMETER

Die einfachste Möglichkeit, Traits als Parameter zu übergeben ist mit der `impl Trait` Syntax.

```
fn use_drink(drink: &impl Drinkable) → Effect {  
    println!("Drinking my drink!");  
    drink.drink()  
}
```

Diese Syntax hat den Nachteil, dass bei mehreren Trait Parameter nicht festlegen kann, dass diese denselben Typ haben müssen. Im Beispiel unten könnte z.B. `empty_drink`: Beer und `full_drink`: Acid sein.

```
fn refill(empty_drink: &impl Drinkable, full_drink: &impl Drinkable)
```

#### 13.4.1. Trait Bound Syntax

`impl Trait` ist nur Syntactic Sugar für die *Trait Bound Syntax*. Mit ihr lässt sich auch das obige Problem lösen; nun müssen beide Parameter vom gleichen Typ sein.

```
fn refill<T: Drinkable>(empty_drink: &T, full_drink: &T)
```

#### 13.4.2. Mehrere Trait Bounds

Mit beiden Syntaxen können auch mehrere Trait Bounds auf Parameter angewendet werden.

```
// impl Trait Syntax                                // Trait Bound Syntax  
fn use_drink(drink: &impl Drinkable + Display)    fn use_drink<T: Drinkable + Display>(drink: T&)
```

Weil dies aber schnell komplex werden kann, gibt es die `where`-Clause Syntax.

```
// Trait Bound Syntax  
fn run<T: Display + Clone, U: Clone + Debug>(t: &T, u: &U) → i32 {  
    // ...  
}  
  
// where clause Syntax  
fn run<T, U>(t: &T, u: &U) → i32  
where  
    T: Display + Clone,  
    U: Clone + Debug  
{  
    // ...  
}
```

### 13.5. TRAITS ALS RÜCKGABETYPEN

Objekte eines Traits können auch aus einer Funktion zurückgegeben werden. Dabei kann irgendein Typ, der den Trait implementiert verwendet werden.

```
fn get_drink() → impl Drinkable {  
    Beer { brand: String::from("Schüga") }  
}
```

Es ist aber nicht möglich, konditionell verschiedene Typen zurückzugeben. Diese Limitation kann aber mit Box umgangen werden, siehe [«Box<T>» \(Seite 32\)](#)

```
// Compiler error: `if` and `else` have incompatible types  
fn get_drink(poisonous: bool) → impl Drinkable {  
    if poisonous {  
        Acid {}  
    } else {  
        Beer { brand: String::from("Calanda") }  
    }  
}
```

### 13.6. METHODEN NACH IMPLEMENTIEREN TRAITS KREIEREN

Manche Methoden sollen nur zur Verfügung stehen, wenn ein Objekt bestimmte Traits implementiert. Im Beispiel unten wird die Methode `new()` für alle möglichen `T`s bereitgestellt, `cmp_and_display()` aber nur, wenn der Typ `T` die Traits `Display` und `PartialOrd` implementiert - was `Integer` tut, `Vector` aber nicht (*implementiert nur `PartialOrd`*).

```
use std::fmt::Display;

struct Pair<T> {
    x: T,
    y: T,
}

impl<T> Pair<T> {
    fn new(x: T, y: T) → Self { Self { x, y } }
}

impl<T: Display + PartialOrd> Pair<T> {
    fn cmp_and_display(&self) {
        if self.x ≥ self.y {
            println!("The largest member is x = {}", self.x);
        } else {
            println!("The largest member is y = {}", self.y);
        }
    }
}
```

```
let pair1 = Pair::new(1, 2);
pair1.cmp_and_display();
// Output:
// The largest member is y = 2

let v1 = vec!(1);
let v2 = vec!(2);
let pair2 = Pair::new(v1, v2);
pair2.cmp_and_display();
// Compiler Error:
// the method `cmp_and_display`
// exists for struct
// `Pair<Vec<{integer}>>`, but its
// trait bounds were not satisfied
```

### 13.7. TRAIT OBJEKTE

Trait Objekte repräsentieren einen beliebigen Typ, welcher einen bestimmten Trait implementiert. Ein Trait Objekt kann gleich wie ein generischer oder spezifischer Typ verwendet werden, ausser dass diese immer auf dem Heap gespeichert werden müssen und mit dem `dyn` Keyword spezifiziert werden müssen.

Angenommen, in einer GUI Library soll es eine Liste von allen Komponenten des GUIs geben (*Buttons, Listen, Bilder etc.*). Wenn das Programm gestartet wird, soll die Library durch diese Liste durchgehen und die Komponenten auf dem Screen zeichnen. Dazu bietet sich ein `Draw` Trait an. Doch welchen Typ soll der Vector haben, der die Elemente enthält? Hier kann ein Trait Objekt verwendet werden

```
pub Trait Draw {
    fn draw(&self);
}

pub struct Screen {
    pub components: Vec<Box<dyn Draw>>,
}

impl Screen {
    pub fn run(&self) {
        for component in self.components.iter() {
            component.draw();
        }
    }
}
```

Die Komponenten müssen mit `Box<T>` auf dem Heap gespeichert werden (siehe Kapitel [«Box<T>» \(Seite 32\)](#)), weil nicht zur Compilezeit bestimmen werden kann, wie gross die Elemente sein können, die `Draw` implementieren. Das `dyn` Keyword wird benötigt, um dem Compiler mitzuteilen, dass *Dynamic Dispatch* auf diesen Elementen angewendet wird: Die zu ausführende `draw`-Funktion jedes Elements wird zur Laufzeit bestimmt.

---

## 14. LIFETIMES

Referenzen zeigen immer auf valide Objekte. Deswegen stellt der Rust Compiler sicher, dass Referenzen immer mindestens so lange leben wie das Objekt dahinter. Der Compiler berechnet also die Lifetime von Referenzen und stellt sicher, dass diese immer eingehalten werden, sodass dangling pointer vermieden werden.

Die Lifetime einer Referenz beginnt bei der Zuweisung und endet bei der letzten Verwendung der Referenz oder beim Verlassen des Scopes, in welchem das referenzierte Objekt definiert ist – je nach dem was früher der Fall ist.

Es gibt zwei Arten, um Lifetimes zu veranschaulichen: *Variabel Scope* und *Referenzen-Lifetimes*. Ersteres wird häufig in Rust-Literatur verwendet, aber letzteres ist als anderer Blickwinkel ebenfalls spannend.

Im Beispiel wird eine Referenz `r` deklariert (ohne Initialisierung, was in Rust eigentlich nicht erlaubt ist – ausser auf die Variable wird vor dem Initialisieren nicht zugegriffen). Im inneren Scope wird dann ein `i32` erstellt und eine Referenz darauf `r` zugewiesen. Der Scope endet dann und der Integer wird gedropped, der Zugriff auf die Referenz `r` in `println!()` ist also ungültig.

Links sieht man den Scope der Variablen, welche mit Referenzen zu tun haben. Zur Illustration wird der Scope von `r` mit `'r'` und von `s` mit `'s'` gekennzeichnet. Da der Scope des referenzierten Objekts `'s'` kürzer ist, als die der Referenz `'r'` kompiliert dieser Code nicht.

Auf der rechten Seite wird am selben Code die Lifetime der Referenz gezeigt. Diese startet, sobald die Referenz zugewiesen wird und endet nach dem letzten Zugriff auf die Referenz im `println!()`. Aus dieser Sicht kompiliert der Code nicht, weil Scope-Grenze überschritten wird, welche das referenzierte Objekt `s` aus dem Scope fallen lässt.

|                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>fn main() {           // Variable scopes   let r;              // -----+ 'r   {     let s = 69;       // --+ 's       r = &amp;s;           //       }                  // --+     println!("{r}");    //   }                    // -----+ // Error: `s` does not live long enough</pre> | <pre>fn main() {           // Reference Lifetime   let r;              // -----+ 'r   {     let s = 69;       // -----+ 'r     r = &amp;s;           //     }                  // --- end of scope     println!("{r}");    // -----+ }                    // // Compiler Error: `s` does not live long enough</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Findet der letzte Zugriff auf eine Referenz im selben Scope statt, ist der Zugriff wieder in Ordnung und der Code kompiliert. Es spielt also nicht nur eine Rolle wo eine Referenz valide ist, sondern auch auf was eine Referenz zeigen kann.

Im Beispiel unten wird das mit der Set-Notation verdeutlicht. Der Code links kompiliert, da die Lifetime von `r` unterbrochen wird, weil zwischen dem Verlassen des Scopes und der Neuweisung von `r` nicht auf `r` zugegriffen wird. Anders sieht es rechts aus: Da die Zuweisung nun konditionell ist, kann sowohl das gültige Objekt `n`, als auch das nun ungültige Objekt `s` referenziert sein. Dieser Code kompiliert damit nicht.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>fn main() {           // Lifetimes don't overlap   let n = 420;         // Compiler is happy :)   let mut r;          //   {     let s = 69;        //     r = &amp;s;            // ---+ 'r = { &amp;s }                       //       println!("{r}");   // ---+   }                  // --- end of scope                       //   r = &amp;n;              // ---+ 'r = { &amp;n }                       //     println!("{r}");     // ---+ }                    //</pre> | <pre>fn main() {           // Compiler Error:   let n = 420;         // `s` does not live   let mut r;          // long enough   {     let s = 69;        //     r = &amp;s;            // ---+ 'r = { &amp;s }                       //       println!("{r}");   // ---+   }                  // --   -- end of scope   if random_bool {    //       r = &amp;n;            // ---+ 'r = { &amp;s, &amp;n }   }                  //     println!("{r}");     // ---+ }                    //</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Diese Beispiele sind künstlich erzeugt, zeigen aber gut auf, wie Lifetimes funktionieren. Schauen wir nun, wie Lifetimes in «echtem» Code funktionieren.

## 14.1. LIFETIME ANNOTATIONS IN FUNKTIONEN

Wir schreiben eine Funktion, welche den längeren von zwei Strings zurückgibt. Da die Funktion keine Ownership benötigt, übergeben wir die Parameter als String slices.

```
fn longest(str1: &str, str2: &str) → &str {
  if str1.len() > str2.len() { str1 } else { str2 }
} // Compiler Error: missing lifetime specifier. this function's return type contains a borrowed
// value, but the signature does not say whether it is borrowed from `str1` or `str2`
```

Der Compiler weiss also, dass wir entweder `str1` oder `str2` zurückgeben. Das Problem ist allerdings, dass die referenzierten Objekte wie in den oberen Beispielen in anderen Scopes sein können. Der Compiler kann dies aber (momentan) nicht überprüfen, weil er nicht sicher ist, wie die Parameter und der Rückgabetyt im Verhältnis zueinander stehen.

Lifetimes sind meistens implizit, aber in Fällen wie diesen müssen sie explizit angegeben werden. Das geschieht mit der **Lifetime Annotation Syntax**, die ähnlich funktioniert wie bei Generics und Traits. Jede Lifetime hat ihren eigenen Identifier, welcher mit « ' » beginnt – die Konvention ist es, die Lifetimes 'a, 'b etc. zu nennen. Lifetimes auf Parametern sind **input lifetimes**, auf Rückgabetypen **output lifetimes**.

```
fn longest<'a>(str1: &'a str, str2: &'a str) → &'a str {
    if str1.len() > str2.len() { str1 } else { str2 }
}
```

Die Funktion oben wurde nun mit Lifetime Annotations versehen und sagt folgendes aus: **Die Lifetime 'a des Return Types muss mindestens genau so lang sein, wie die kleinere Lifetime der beiden Parameter str1 und str2.**

```
fn main() {
    let string1 = String::from("longer string");
    let result;
    {
        let string2 = String::from("smol");
        result = longest(&string1, &string2);
    }
    println!("The longest string is {result}");
}

// Does not compile: Lifetime of &string2 is
// smaller than lifetime of &result
//
//
// ----- 'result = { &string1, &string2 }
//      |
// --- | -- end of scope
//      |
// -----+

fn main() {
    let string1 = String::from("longer string");
    {
        let string2 = String::from("smol");
        let result = longest(&string1, &string2);
        println!("The longest string is {result}");
    }
}

// Compiles: Lifetime of &string2 is equal to
// lifetime of &result
//
//
// ----- 'result = { &string1, &string2 }
//      |
// -----+
//
// ----- end of scope
```

## 14.2. NOW YOU'RE THINKING WITH LIFETIMES!

Die Lifetime muss nur explizit angegeben werden, wenn eine Funktion mehr als einen Referenzparameter besitzt und einer von diesen zurückgeben kann. Die untenstehende Funktion `find_first()` durchsucht jede Zeile von `content` nach einem Muster `query` und gibt die erste Zeile von `content` zurück, auf welcher `query` gefunden wurde.

`find_first()` hat also nur eine Lifetime Annotation für `content` und den Return Type, nicht aber für den Parameter `query`, welcher nie zurückgegeben werden kann.

```
fn find_first<'a>(query: &str, content: &'a str) → &'a str { // No lifetime annotation
    for line in contents.line() { // on `query`...
        if line.contains(query) {
            line // ...because only (part of)
        } // `content` can be returned
    }
}
```

Ebenfalls wäre theoretisch eine Lifetime Annotation für einen einzigen Referenzparameter nötig. Rust kann diesen aber inferren; durch das Weglassen wenden wir **lifetime elision** an.

```
fn return_parameter<'a>(s: &'a str) → &'a str { // Lifetime annotation unnecessary, can be
    s // inferred by the compiler
}
```

Eine weitere nützliche Denkweise ist, dass output lifetimes ein Superset der input lifetimes sind; die Lifetime der Rückgabewerte muss also die Lifetime aller (explizit spezifizierten) Parameter-Lifetimes enthalten. Im Beispiel unten sind die Lifetimes der Strings explizit angegeben. Diese sind hier nur zur Veranschaulichung da, damit der Code kompiliert müssten sie noch in der `main()`-Signatur genauer spezifiziert werden.

Beim ersten Aufruf von `longest` könnte entweder `b` oder `g` zurückgegeben werden – die Lifetime `'l1` muss also mindestens so lange wie `'b` und `'g` sein. Beim zweiten Aufruf muss die Lifetime `'l2` mindestens so lange leben wie `'l1` und `'v`. `'l2` beinhaltet also `'v` und `'l1`, welche `'b` und `'g` beinhaltet. Oder anders gesagt: `'l2` muss mindestens genauso lange leben wie `b`, `g` und `v`.

|                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                              |                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <pre>fn main() {   let b: &amp;'b str = "Bier";   let g: &amp;'g str = "Gin";   let v: &amp;'v str = "Vodka";    let l1: &amp;'l1 str = longest(b, g);   let l2: &amp;'l2 str = longest(l1, v);    println!("{l2} is longest"); }  fn longest&lt;'a&gt;(x: &amp;'a str, y: &amp;'a str) → &amp;'a str {   if x.len() &gt; y.len() { x } else { y } }</pre> | <pre>// +-----+ //   'l2 +-----+   //     'l1 +-----+     //     'b   "Bier"       //     +-----+     //     'g   "Gin"       //     +-----+     //   +-----+     //   +-----+     //   'v   "Vodka"       //   +-----+     // +-----+</pre> | $l_1 = \{b, g\}$<br>$l_2 = \{l_1, v\}$<br>$l_2 \supset l_1$ |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|

Wichtig ist: Lifetime Annotations ändern nicht, wie lange eine Referenz lebt – sie zeigen nur die **Beziehung verschiedener Lifetimes zueinander** auf. Nur weil alle Referenzen diesselbe Lifetime Annotation haben, bedeutet das nicht, dass sie die exakt selbe Lifetime haben müssen.

Deutlich wird das, wenn wir jeder Referenz eine eigene Lifetime Annotation geben: `'s1` und `'s2` für die Parameter und `'out` für den Return Type. Nun müssen wir aber die verschiedenen Lifetimes mithilfe eines **outlives constraints** miteinander verknüpfen. Im Beispiel unten sagen sie aus, dass `'s1` und `'s2` ein Subset von `'out` sind – also genau das, was wir schon mit der einen Lifetime `'a` erreicht haben.

```
fn longest<'s1, 's2, 'out>(s1: &'s1 str, s2: &'s2 str) → &'out str
  where 's1: 'out, 's2: 'out
```

### 14.3. DIE STATIC LIFETIME

Stringlitterale haben eine spezielle Lifetime `static`, welche aussagt, dass die Referenz für die gesamte Laufzeit des Programms gültig ist. Da Stringlitterale direkt im Binary gespeichert werden, ist diese Aussage korrekt – die Referenz geht nie aus dem Scope, weil theoretisch jede Funktion immer auf diese zugreifen kann.

## 15. TESTING

Rust hat native Unterstützung für Unit Tests. Wie bei vielen anderen Sprachen soll auch bei Rust so wenig Code wie möglich in der `main()`-Funktion stehen, da dieser nicht getestet werden kann. Die Tests der einzelnen Funktionen werden per Konvention direkt in der selben Datei wie der zu testende Code geschrieben.<sup>7</sup> Dazu wird ein neues Modul erstellt und versieht dieses mit der `#[cfg(test)]`-Annotation – damit wird dieser Code nur beim Builden und Ausführen der Test-Konfiguration verwendet. Die einzelnen Testfunktionen werden mit `#[test]` annotiert.

Die Testresultate können mit den `assert*!()`-Makros verifiziert werden (`assert!()`, `assert_eq!()`, `assert_ne!()`). Schlägt der Test fehl, rufen sie `panic!` auf. Um die Makros verwenden zu können, müssen die verwendeten Typen die **PartialEq und Debug Traits** implementieren, was aber simpel ist, da sich beide mit `#[derive(PartialEq, Debug)]` derivieren lassen.

<sup>7</sup><https://doc.rust-lang.org/stable/book/ch11-03-test-organization.html>

```
fn square(n: i32) → i32 {
    n * n
}

#[cfg(test)]
mod tests {
    use super::*; // Bring the main functions into scope of tests module

    #[test]
    fn number_is_squared() {
        let result = square(16);
        assert_eq!(result, 256);
    }
}
```

Die Tests werden mit Befehl `cargo test` ausgeführt.

```
$ cargo test
Compiling square v0.0.1 (file:///test-da-tests)
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.80s
Running unittests src/lib.rs (target/debug/deps/playground-1b25adf163e1ce97)
```

```
running 1 test
test tests::number_is_squared ... ok
```

```
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

```
Doc-tests square
```

```
running 0 tests
```

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

Alle weiteren Argumente der `assert!()` Makros werden `format!()` übergeben und angezeigt, wenn ein Test fehlschlägt.

```
let result = "actual test result";
assert!(result.contains("Test"), "Result didn't contain 'Test', value was {result}");
```

## 15.1. TESTS AUF PANIC!() ÜBERPRÜFEN

Es können auch Tests geschrieben werden, welche auf `panic!()` prüfen, indem der Test mit dem Attribut `#[should_panic]` versehen wird.

|                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#[derive(Debug, PartialEq)] struct DiceRoll {     value: u8 }  impl DiceRoll {     fn new(value: u8) → DiceRoll {         if value &lt; 1    value &gt; 6 {             panic!("Dice roll not between 1 and 6, got {value}");         }         DiceRoll { value }     } }</pre> | <pre>#[cfg(test)] mod tests {     use super::*;      #[test]     fn value_is_equal() {         let result = DiceRoll::new(2);         assert_eq!(result, 2);     }      #[test]     #[should_panic]     fn invalid_dice() {         DiceRoll::new(420);     } }</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

should\_panic gibt bei jeder Art von Panic true aus. Soll die panic!()-Art genauer spezifiziert werden, kann dem Attribut ein Teil der entsprechenden panic!()-Nachricht angefügt werden.

```
impl DiceRoll {
    fn new(value: u8) → DiceRoll {
        if value > 100 {
            panic!("Value is not ≥ 6, got {value}");
        } else if value < 1 {
            panic!("Zero is invalid");
        }
        DiceRoll { value }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    #[should_panic(expected = "is not ≥ 6")]
    fn greater_than_6() {
        DiceRoll::new(420);
    }
}
```

## 15.2. TESTS MIT RESULT<T, E>

Anstatt panic!() aufzurufen, kann ein Test auch Result returnen. Damit kann der Try Operator verwendet werden, um einen Test fehlschlagen zu lassen, wenn irgendwo im Code ein Err zurückgegeben wird. Der Typ T im Ok kann dabei ein Unit Type sein, da beim Erfolg eines Tests nichts zurückgegeben wird.

So kann aber should\_panic nicht mehr verwendet werden. Um Funktionen, die Result zurückgeben, auf Err zu prüfen, wird assert!(value.is\_err()) empfohlen.

```
#[test]
fn result_test() → Result<(), String> {
    let result = square(16);
    if result == 256 {
        Ok(())
    } else {
        Err(String::from("16^2 is not 256"))
    }
}

#[test]
fn should_return_err() {
    let result = func_returns_result();
    assert!(result.is_err());
}
```

## 15.3. INTEGRATION TESTS

Per Konvention werden Integration Tests in einem /tests-Ordner im Root des Projektes abgelegt. Die Dateien in diesem Ordner werden automatisch als Tests behandelt, darum ist #[cfg(test)] nicht nötig. Die zu testenden Funktionen müssen aber in den Scope gebracht werden.

```
mycrate
├── Cargo.lock
├── Cargo.toml
├── src
│   └── lib.rs
└── tests
    └── integration_test.rs

use mycrate::square;

#[test]
fn squares() {
    let result = square(2);
    assert_eq!(result, 4);
}
```

Helper-Funktionen werden in einem separaten Unterordner in einer mod.rs-Datei abgelegt. Sie können dann mit mod common; importiert werden.

```
mycrate
├── Cargo.lock
├── Cargo.toml
├── src
│   └── lib.rs
└── tests
    ├── common
    │   └── mod.rs
    └── integration_test.rs

// mod.rs
pub fn setup() { /* integration test setup */ }

// integration_test.rs
use mycrate::square;
mod common;
#[test]
fn squares() {
    common::setup();
    let result = square(2);
    assert_eq!(result, 4);
}
```

---

## 16. CODE-DOKUMENTATION

Funktionen können mit **documentation comments** beschrieben werden. Ihr Inhalt verwendet Markdown. Sie werden mit «`///`» vor der entsprechenden Funktion platziert

```
/// Adds one to the number given.
///
/// # Examples
/// ```
/// let arg = 5;
/// let answer = my_amazing_math_crate::add_one(arg);
///
/// assert_eq!(6, answer);
/// ```
pub fn add_one(x: i32) → i32 {
    x + 1
}
```

Soll ein Kommentar für das Element, welches die documentation comments enthält gelten (z.B. eine Beschreibung der Crate), wird `//!` verwendet.

```
//! # My amazing math crate
//!
//! `my_amazing_math_create` is a collection of utilities to make performing certain
//! calculations more convenient.
```

Mit dem Befehl `cargo doc` kann aus diesen Kommentaren **automatisch eine Dokumentation in HTML** für diese Crate generiert werden.

Ein weiteres nützliches Feature ist, dass Codebeispiele unter dem «`# Examples`»-Header **automatisch mit cargo tests ausgeführt** werden. Damit bleiben die Beispiele in der API-Dokumentation immer aktuell.

---

## 17. SMART POINTERS

Mit Smart Pointer können Daten auf dem Heap anstatt auf dem Stack abgelegt werden. Die meisten Smart Pointer sind auch Owner von den Daten auf dem Stack, im Gegensatz zu Referenzen, die nur Borrowing betreiben. Smart Pointer übernehmen das Allokieren und Destruieren auf dem Heap und implementieren so das RAII-Prinzip (*Resource Acquisition is Initialization*).

Smart Pointer sind ein fortgeschrittenes Thema und werden nur in spezifischen Bereichen benötigt. Der Vollständigkeit halber werden sie hier aber trotzdem kurz erklärt.

### Wieso speichert man Daten auf dem Heap?

Idealerweise werden alle Daten im Stack gespeichert, da das Allokieren, Zugreifen und Deallozieren von Werten dort schneller ist. Das ist aber aus verschiedenen Gründen nicht immer möglich.

1. Alle Werte (*beziehungsweise ihre Typen*) auf dem Stack müssen zur Compilezeit eine feste und unveränderbare Grösse haben. Ist dies nicht möglich, weil sich die Grösse ändern kann (z.B. bei *String* oder *Vec*) oder die Grösse zur Compilezeit nicht ermittelt werden kann, müssen diese Daten auf dem Heap gespeichert werden
2. Der Stack wird beim Verlassen einer Methode abgeräumt. Soll auf Daten methodenübergreifend zugegriffen werden, müssen diese also auf dem Heap gespeichert werden.

Für verschiedene Einsatzzwecke gibt es verschiedene Arten von Smart Pointern.

- **Box<T>**: Speichert Werte auf dem Heap, ohne weitere Spezialfunktionalität
- **Rc<T>**: Ein Reference Counter, der mehrere Owner erlaubt
- **Arc<T>**: Wie Rc<T>, aber thread-safe
- **Mutex<T>**: Erlaubt atomaren Zugriff auf Daten
- **Ref<T>/RefMut<T>**: Lässt die Borrowing-Regeln zur Laufzeit anstatt zur Compilezeit prüfen. Wird über `RefCell<T>` zugegriffen

### 17.1. BOX<T>

Der simpelste Smart Pointer `Box<T>` speichert Werte auf dem Heap anstatt auf dem Stack, ohne sonstige Funktionalitäten. Sie besteht aus den Daten auf dem Heap und dem Pointer auf diese Daten, welcher auf dem Stack liegt. Es gibt zwei Hauptanwendungen für eine Box:

#### Rekursive Typen

Bei Typen, welche sich selber enthalten, kann die Grösse des Typen nicht zur Compilezeit festgestellt werden, da sie theoretisch unendlich gross sein können. Solche rekursive Typen müssen in eine Box gepackt werden, da eine Box eine konstante Grösse hat.

```
enum LinkedList {
    Element(i32, Box<LinkedList>),
    None
}
```

```
let link = LinkedList(1, LinkedList(2, LinkedList(None)));
```

#### Trait-Variablen

Variablen mit einem Trait als Typ kann die Grösse ebenfalls nicht zur Compilezeit festgestellt werden, da jede Implementation eines Traits eine andere Grösse haben kann. Ihr Inhalt muss deswegen ebenfalls geboxed werden.

```
trait Vehicle {
    fn drive(&self);
}
struct Car;
impl Vehicle for Car {
    fn drive(&self) {
        println!("Wroom Wroom!");
    }
}
```

```
// Does not compile:
// the size for values of type `dyn Vehicle`
// cannot be known at compilation time
let vehicle: dyn Vehicle;
vehicle = Truck;
// Correct:
let vehicle: Box<dyn Vehicle>;
vehicle = Box::new(Truck);
vehicle.drive();
```

### 17.2. RC<T>

Rc steht für **Reference Counter**. Dieser Typ zählt die Anzahl Referenzen auf seinen Wert und löscht diesen erst, wenn keine Referenzen mehr vorhanden sind. Rc erlaubt also «mehrere Owner» von Daten, aber anstatt die Ownership zu übergeben, wird Rc geklont, was die Anzahl Referenzen erhöht. Verlässt ein Rc den Scope, wird die Anzahl Referenzen um eins dekrementiert. Sind keine Referenzen mehr vorhanden, wird der Wert innerhalb vom Rc ebenfalls gedropped.

Damit ist es beispielsweise möglich, dass im Beispiel der `LinkedList` vom Box-Beispiel mehrere `LinkedList` gleichzeitig eine bestimmte Liste beinhalten.

```
enum LinkedList {
    Element(i32, Rc<LinkedList>),
    None
}
```

```
let a = Rc::new(LinkedList(7, Rc::new(LinkedList(None))));
let b = LinkedList(3, Rc::clone(&a));
let c = LinkedList(1, Rc::clone(&a));
```

### 17.3. MUTEX<T>

Um Daten vor gleichzeitigem Zugriff und damit Data Races zu schützen, hat Rust ebenfalls Mutexes. Während das Locking explizit ist, geschieht das Unlocking automatisch am Ende des Scopes.

```
let m = Mutex::new(5);
{
    let mut num = m.lock().unwrap();
    *num = 6;
} // Mutex automatically unlocked here
println!("m = {m:?}");
```

### 17.4. ARC<T>

Arc hat grundsätzlich dieselbe Funktionalität wie Rc, ist aber thread-safe und kann somit in multithreaded Kontexten verwendet werden. Dadurch ist die Implementation aber langsamer als Rc.

### 17.5. REFCELL<T> UND INTERIOR MUTABILITY

Mit *Interior Mutability* können Daten geändert werden, auch wenn nur immutable Referenzen vorhanden sind. `RefCell<T>` verwendet dabei «unsafe», um Rust's Borrow Checker zu umgehen. Darum sollte Interior Mutability nur angewandt werden, wenn die Borrow-Regeln auch zur Runtime eingehalten werden.

Als Beispiel für Interior Mutability nehmen wir ein Mock-Objekt für das Testing. Es soll den Messenger-Trait implementieren, welcher im echten Code Nachrichten übers Internet sendet. Der MockMessenger soll aber stattdessen alle Nachrichten in einem Vec speichern, damit Asserts dagegen ausgeführt werden können.

```
pub trait Messenger {
    fn send(&self, msg: &str);
}

struct MockMessenger {
    sent_messages: Vec<String>,
}
```

Wir implementieren den Trait und schreiben einen Test Case für das Verhalten.

```
impl Messenger for MockMessenger {
    fn send(&self, msg: &str) {
        self.sent_messages.push(String::from(msg));
    }
} // Compiler Error: Cannot borrow
// `self.sent_messages` as mutable, as it is
// behind a `&` reference

#[test]
fn test_send_messages() {
    let mock_msgr = MockMessenger::new();
    mock_msgr.send("Test Message");
    assert_eq!(mock_msgr.sent_messages.len(), 1);
}
```

Der Compiler sagt uns, dass wir `sent_message` nicht verändern können, weil der `&self`-Parameter in `send()` immutable ist. Wir könnten das theoretisch ändern, aber dann müssten wir für unsere Testcases die «echten» Methoden anpassen, was nicht wünschenswert ist. Hier kommt also Interior Mutability zum Zug!

Der `sent_messages` Vector wird in eine `RefCell` gepackt, um ihn manuell borrowen zu können. In der `send()`-Methode können wir jetzt über die immutable `self`-Referenz den Vec manuell mutable borrowen und die gesendete Nachricht hinzufügen. Im Test Case wird der Vec für den Assert manuell immutable borrowed, um ihn auslesen zu können. Wir benutzen also `RefCell`, um den inneren Wert `sent_messages` zu mutieren, während der äussere Wert `MockMessenger` immutable ist.

```
struct MockMessenger {
    sent_messages: RefCell<Vec<String>>,
}

impl Messenger for MockMessenger {
    fn send(&self, msg: &str) {
        self.sent_messages.borrow_mut()
            .push(String::from(msg));
    }
}

#[test]
fn test_send_messages() {
    let mock_msgr = MockMessenger::new();
    mock_msgr.send("Test Message");
    assert_eq!(
        mock_msgr.sent_messages.borrow().len(),
        1
    );
}
```

### 17.5.1. Mehrere Owner von Mutable Daten

RefCell wird oft in Kombination mit Rc verwendet. Rc lässt mehrere Owner auf Daten zu, gibt aber nur immutable Zugriff. Packt man RefCell nun in eine Rc, kann man mehrere Owner und mutierbare Daten gleichzeitig haben.

Wir können unsere LinkedList ein weiteres Mal ergänzen. Element besteht nun aus einer RefCell innerhalb eines Rc. Damit kann nun der Wert eines Element bearbeitet werden.

```
#[derive(Debug)]
enum LinkedList {
    Element(Rc<RefCell<i32>>, Rc<LinkedList>),
    None,
}

let value = Rc::new(RefCell::new(5)); // Create a value
let a = Rc::new(Element(Rc::clone(&value), Rc::new(None))); // Insert value
let b = Element(Rc::new(RefCell::new(3)), Rc::clone(&a)); // Insert List into other lists
let c = Element(Rc::new(RefCell::new(3)), Rc::clone(&a));
*value.borrow_mut() += 10; // Update value

println!("a after = {a:?}");
println!("b after = {b:?}");
println!("c after = {c:?}");
// a after = Element(RefCell { value: 15 }, None)
// b after = Element(RefCell { value: 3 }, Element(RefCell { value: 15 }, None))
// c after = Element(RefCell { value: 3 }, Element(RefCell { value: 15 }, None))
```

### 17.6. REFERENZZYKLEN MIT WEAK<T> VERMEIDEN

Rust macht es schwieriger, aber nicht unmöglich, Speicherbereiche zu erstellen, die nie wieder aufgeräumt werden: Ein **Memory Leak**. Aber immerhin sind Memory Leaks in Rust Memory Safe 😊. Mit Rc und RefCell ist es möglich, dass sich zwei Elemente gegenseitig referenzieren, was dazu führen kann, dass sie nie gedropped werden, weil ihr Referenz-Counter nie 0 wird – ein Referenzzyklus.

Rc haben neben den bisher besprochenen Strong References auch Weak References. Während mit Strong References Ownership geteilt werden kann, zeigen Weak References eine Beziehung ohne Ownership auf. Elemente werden gedropped wenn die Anzahl Strong References 0 erreicht, egal wie viele Weak References dann noch vorhanden sind.

Da das Element hinter einer Weak Referenz theoretisch bereits gedropped sein kann, muss vor dem Zugriff immer geprüft werden, ob das Element noch existiert. Dazu muss upgrade() aufgerufen werden, welche Option<Rc<T>> returnt. Die Weak Reference wird also auf eine Strong Reference upgraded.

Im folgenden Beispiel bauen wir einen Tree auf. Ein Node soll mutable Ownership über seine Child-Nodes haben und die Child Nodes sollen Zugriff auf ihren Parent Node haben. Wir können nicht für beide Referenzen Rc<T> verwenden, da das zu einem Referenzzyklus führen würde. Es ist aber sinnvoll, dass der Parent Ownership über seine Children hat: Wird ein Parent gedropped, sollen alle seine Children ebenfalls gedropped werden. Wird ein Child gedropped, soll der Parent immer noch weiterleben – ein idealer Einsatzzweck für Weak References!

```
#[derive(Debug)]
struct Node {
    value: i32,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

fn main() {
    let leaf = Rc::new(Node {
        value: 3,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![]),
    });
```

```
// To access the parent borrow the `parent` RefCell and upgrade the Weak to a Strong Ref
println!("leaf parent = {:?}", leaf.parent.borrow().upgrade());
// Output: leaf parent = None

let branch = Rc::new(Node {
    value: 5,
    parent: RefCell::new(Weak::new()),
    children: RefCell::new(vec![Rc::clone(&leaf)]), // Set leaf as child node
});

// Set a weak reference with `Rc::downgrade` in `leaf` to `branch`
*leaf.parent.borrow_mut() = Rc::downgrade(&branch);

println!("leaf parent = {:?}", leaf.parent.borrow().upgrade());
// Output (`branch`, `children` of branch, `children` of leaf):
// leaf parent = Some(Node { value: 5, parent: RefCell { value: (Weak) },
// children: RefCell { value: [Node { value: 3, parent: RefCell { value: (Weak) },
// children: RefCell { value: [] } } ] } })
```

## 18. MACROS

Wie bereits an vielen Beispielen gesehen, werden in Rust häufig Macros verwendet. Sie werden zur Compilezeit in Rust Code umgewandelt und können so die Syntax von Rust erweitern. Ihr Identifier endet immer in einem «!». Sie sind sehr nützlich, um Boilerplate-Code wegzuschaffen.

Es gibt mehrere Arten von Makros:

- **Deklarative Makros:** Werden mit `macro_rules!` definiert. Erlauben das Matchen von Rust Code
- **Derive Makros:** Erlauben das Implementieren von eigenen `#[derive]`-Attributen auf Structs und Enums
- **Attribut-Makros:** Erlauben das Implementieren von komplett eigenen Attributen, die auf sämtliche Rust-Elemente angewendet werden können
- **Funktions-Makros:** Sehen aus wie reguläre Funktionen, verarbeiten aber die Tokens ihrer Argumente anstatt ihrer Werte. Ähnlich wie deklarative Makros, können aber mehr als nur Matchen der Argumente.

Die letzten drei Arten sind **prozedurale Makros** und operieren mit Tokens, ähnlich wie ein Compiler. Solche Makros sind relativ komplex und werden darum hier nicht weiter behandelt. Sie sind sehr mächtig, beispielsweise kann die `sqlx` Crate SQL-Queries zur Compilezeit auf einer Datenbank ausführen, um so die SQL-Syntax und das Vorhandensein entsprechender Tabellen und Spalten überprüfen. Schlägt die Query fehl, wird der Datenbank-Fehler direkt als Compilerfehler in Rust angezeigt.

```
let name = sqlx::query!(SELECT name, FROM user WHERE id=1).fetch_one(&mut con).await?;
let error = sqlx::query!(WELECT * from user);
// Compiler Error: error returned from database: near "WELECT": syntax error
```

### 18.1. DEKLARATIVE MAKROS

Macros selbst zu schreiben ist am Anfang eine Herausforderung, da sie wie eine eigene Sprache in der Sprache sind. Wir schauen uns hier aber kurz ein eigenes, simples Makro an, welches die Grundrechenoperationen für uns ausführt und die Eingabe als String zurückgibt.

Eine deklarative Makro-Definition startet immer mit `macro_rules!`, gefolgt von dem Namen des Makros. Das Attribut `#[macro_export]` bringt das Makro in den Scope der Crate, in welcher es definiert ist. Danach folgt ein **Macro-Pattern** ähnlich wie in einer `match`-Expression. Ein Pattern befindet sich immer innerhalb runder Klammern. In diesen können dann **Macro-Variablen** mit `$` definiert werden. Anders als reguläre Variablen enthalten diese keine Werte, sondern Rust-Code. Auch Makro-Variablen haben Typen, in unserem Fall `expr`, was jede beliebige Rust-Expression `matched`.

Der Code, welcher ausgeführt wird, wenn das Pattern matched, befindet meist innerhalb zwei geschwungener Klammern: Die äusseren sind Teil der Definition des Macros, die inneren werden zur Compilezeit in den tatsächlichen Code hineingeschrieben. Dadurch wird ein neuer Scope geöffnet, um möglichen Namenskonflikten o.ä. vorzubeugen.

Innerhalb des Macrocodes kann regulärer Rust Code geschrieben werden. Zusätzlich kann auch auf die Macro Variablen zugegriffen werden. In unserem Beispiel wird je nach Operator die entsprechende Rechenoperation ausgeführt und ein String mit der Operation und dem Resultat zurückgegeben.

```
#[macro_export]
macro_rules! math_operations {
    ($a: expr, $op: expr, $b: expr) => {{
        let res = match $op {
            "+" => $a + $b,
            "-" => $a - $b,
            "*" => $a * $b,
            "/" => if $b != 0 { $a / $b }
                else { panic!("Division by zero") },
            _ => panic!("Bad operator: {}", $op),
        };
        format!("{}", $a, $op, $b, res)
    }}
}
```

```
// Example usage
pub fn main() {
    math_operations!(4, "+", 2); // 4 + 2 = 6
    math_operations!(10, "-", 3); // 10 - 3 = 7
    math_operations!(6, "*", 4); // 6 * 4 = 24
    math_operations!(15, "/", 3); // 15 / 3 = 5
}
```

Als weiteres Beispiel ist hier eine leicht vereinfachte Variante des `vec!` Makros: Das Pattern sieht ähnlich aus wie oben, hat aber einige Unterschiede. Zuerst wird mit `$( )` ein Unter-Pattern definiert, dann darin eine Makro-Variabel `$x`, welche eine Rust-Expression annimmt. Darauf folgt ein Komma als Separator zwischen den Argumenten und ein `*`, welcher aussagt, dass das vorherige Pattern null- oder mehrmals vorkommen kann. Bei einem Aufruf mit `vec![1, 2, 3];` wird `$x` mit den drei Expressions 1, 2 und 3 gematched.

Im Body des Makros wird zuerst ein mutable Vector erstellt. Dann wird die `push()`-Funktion in `$( )*` so viel mal generiert, wie das Pattern gematched wurde. Bei drei Argumenten wird der Code also dreimal generiert. Danach wird der Vector zurückgegeben.

```
#[macro_export]
macro_rules! vec {
    ( $( $x:expr ),* ) => {
        {
            let mut temp_vec = Vec::new();
            $(
                temp_vec.push($x);
            )*
            temp_vec
        }
    };
}
```

```
// This macro call...
let vector = vec![1, 2, 3];
// ...generates this code at compile time
let vector = {
    let mut temp_vec = Vec::new();
    temp_vec.push(1);
    temp_vec.push(2);
    temp_vec.push(3);
    temp_vec
}
```

## 18.2. NÜTZLICHE MAKROS DER STANDARD LIBRARY

Nachfolgend befinden sich einige Macros der Standard Library, die wir bisher noch nicht gesehen haben (die Verwendung von `println!()` und `panic!()` dürfte mittlerweile bekannt sein 😊)

| Macro                        | Beschreibung                                                                                                                              |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dbg!()</code>          | Gibt den Wert einer Expression für quick-and-dirty Debugging aus<br><code>let x = 3; dbg!(x * 4);</code> // [src/main.rs:2:16] x * 4 = 12 |
| <code>eprintln!()</code>     | Gibt eine Nachricht auf stderr aus. Ideal für Fehlermeldungen.                                                                            |
| <code>debug_assert!()</code> | Führt Assert nur in dev Build durch, nicht aber in release Builds                                                                         |

| <i>Macro</i>                  | <i>Beschreibung</i>                                                                                                                                                                            |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>todo!()</code>          | Lässt noch nicht fertige Funktionen kompilieren. Löst beim Erreichen <code>panic!()</code> mit der Nachricht «not yet implemented» aus. Gedacht für Funktionen, die noch implementiert werden. |
| <code>unimplemented!()</code> | Lässt unimplementierte Funktionen kompilieren. Löst beim Erreichen <code>panic()</code> mit der Nachricht «not implemented» aus. Gedacht für Funktionen, die nie implementiert werden.         |
| <code>include_str!()</code>   | Lädt zur Compilezeit den Inhalt der angegebenen Datei und speichert ihn als <code>&amp;str</code> .                                                                                            |